FINAL Report

ECE 496 2019 - 2020: Design Project
Team 2019155

Client: Plan International Canada
Supervisor: Professor Olivier Trescases

Rafay ul Ebad Khan
KaiWen Ying
Wenxuan You

EXECUTIVE SUMMARY

Energy shortage has always been a problem in numerous refugee camps throughout Africa. Under the supervision of Professor O. Trescases at the University of Toronto, a group of multidisciplinary students have developed a microelectronic grid to serve as a potential solution to this problem, however, the prototype design still requires enhancements before it can be considered for applications in real world scenarios.

Our aim is to design an improved microelectric grid. This report will highlight the goals, detailed metrics, methods of assessment, and the detailed blueprint of the design; any alternative designs will also be included.

Our goal is to make major improvements on the previously established design. The project aims to optimize the efficiency of power transfers, increase the stability of connections between nodes/devices, and refine user experience on the controller interface. Furthermore, the project will introduce a more advanced virtual currency system.

A Financial plan, which includes the cost breakdowns for the implementation of the design in a large scale context. Through this project, we hope to deliver an affordable energy system solution to areas of poverty.

Group Highlights and Individual Contributions

Kaiwen Ying

Work on improving the Android controller application, collaborated with Arthur to develop the overall control system.

1. Created page to browse transaction history.
2. Graphical representation of transaction and battery level history. Designing database to store user data.
3. Developing auto-transfer algorithms.
4. Restyling charts and interfaces for better user experience.
5. Improving robustness of application by implementing better practices that were not present in the previous version, such as better exception handling.
6. Worked with Arthur to finetune the overall control system.

Rafay Khan

Worked on the Feedback Controller initially with Israel, then designed Electrical System to provide power to the user and within the grid to the other system

1. Deciding on using FPGA to design the controller.
2. Designing the Dual Active Bridge on MATLAB.
3. Implement system to provide power to the user
4. Decide on load and tests to make sure it meets requirements for the user
5. Implement Battery Charger so that power can be transferred between the 2 systems.

Wenxuan You (Arthur)

Worked on the Arduino controller, collaborated with Kevin to develop the overall control system.

Connect Arduino board with APP

1. From hardware level (See Figure 4).
2. Fix and implement the bluetooth module.
3. Implement the function that we can get a time-to-time battery-level data (from current sensor)

Connect with the converter-battery system (every household)

1. Implement the switch module, either control the solar charger or between households

Acknowledgement

We would like to thank our supervisor Professor O. Trascases and administrator Professor P. Anderson.

Also, we appreciate the help from the previous team, including Yijia Wang, Rudan Zheng, Wanling Jiang and also Samuel Harrison.

Finally, we have to say thank you for the huge support from the PHD student, Zhe Gong.

TABLE OF CONTENTS

Executive Summary

Group Highlights and Individual Contributions

Acknowledgements

Table of Contents

Introduction

- *Background and motivation*
- *Project goal*
- *Project requirements*
- *Validation and acceptance tests*

Possible solutions and design alternatives

Final Design

- *System-level overview*
- *System Block Diagram*
- *Module-level Descriptions and Design*
- *Assessment of Final Design*

Testing and Verification

Summary and Conclusions

References

Appendices

Appendix A: Gantt Chart History

Appendix B: Financial Plan

Appendix C: Validation and Acceptance Tests

Introduction

BACKGROUND AND MOTIVATION

The aim for Refugee camps is to provide temporary safety for people in emergency situations, but these camps have turned into permanent homes [1]. In the case of Bidi Bidi in Northwestern Uganda this has been permanent for many years and holds nearly 285,000 people [2]. With climate change there are projected to be more causing more requirements of refugee camps [3], so we need a design of a Power system that would be with low implementation cost, with high ability to scale and have low running costs. Bidi Bidi does not have the infrastructure or the capital needed to host currently [4].

Plan International is an organisation that aims to provide humanitarian aid to children's rights all over the globe. Plan International Canada has previously worked on a capstone project with University of Toronto to make a household level hardware unit that can provide lighting and cell phone charging. In this project we aim to build on this to provide power architecture with point-to-point energy transactions by providing the software to access and control, as well as the hardware to make it provide the energy requirements.

PROJECT GOAL

We are going to make an improvement based on the existing design. Our primary project goal is the implementation of a Power System. In the whole transmission system, trade of energy is one of the most important parts. Users connect their Android phones to the software controller called Arduino Due and all the data of trade (sell or buy energy) is able to be checked on the App interface directly. These are the project goals:

- Complete software interface
- Ensure modules like bluetooth, switches works in stability
- Assemble the converter and battery(it should have ability to transfer power as a complete household)
- Get real time data of battery level from current sensor
- Provide power that can be used to operate lights, chargers and a fan

PROJECT REQUIREMENTS

The scope of this project is the optimization, design, and testing of an existing design based on mini-/micro-grids to provide electrical services in refugee camps and settlements.

Functional Requirements

The primary function is:

- Optimize electrical energy transmission and Implement energy-trade system

Secondary Functions can be defined as follows:

- ❖ Store renewable energy

Objectives

#	Objective	Reason	Objective goal
O1	Inexpensive	A low implementation cost is preferable	
O2	Durable	Ambient conditions requires a long shelf life	Meet the general principles on designing for durability described in ISO 13823 [5]
O3	Easy to operate	Increase intuitiveness in design software operation and manipulation of the equipment	Meet human-centered design goals described in ISO 13407 [6]
O4	Portable	Easy for human manipulation and transportation	Should be lighter than 14kg for to be acceptable for a single-person lift [7][8]

Constraints

#	Categories	Constraints
C1	Safety	- shall provide insulation on live components of the design to avoid electrical shock as provided in IEC 62109 [9] - shall provide electrical ratings in the equipment as provided in IEC 62109 [9] - shall provide protection against mechanical hazards [9]

TECHNICAL DESIGN

SYSTEM LEVEL OVERVIEW

For our system, we have implemented an electrical system that uses a 1000W AC-DC converter which is connected to a 12V battery. The grid consists of a single bus which would be connected through current relays. Energy in the system is transferred to the batteries using a 12V 3A AC/DC battery charger. The relays are operated through an arduino system that takes input by the android app which communicates through a bluetooth connection. There is another relay in each household in order to avoid overcharge of battery. The app provides the user with current battery levels of their battery and setup transactions with other units in the system either automatically according to their previous usage and buying history or manually according to new requirements.

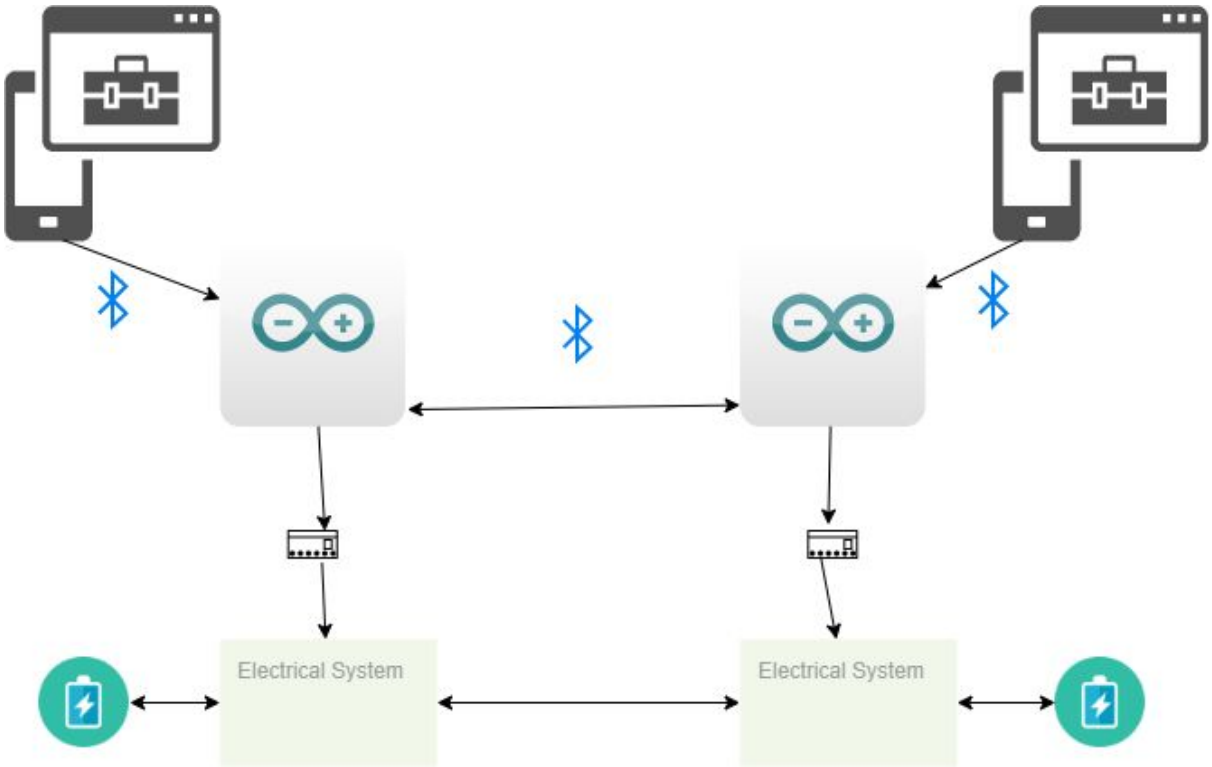
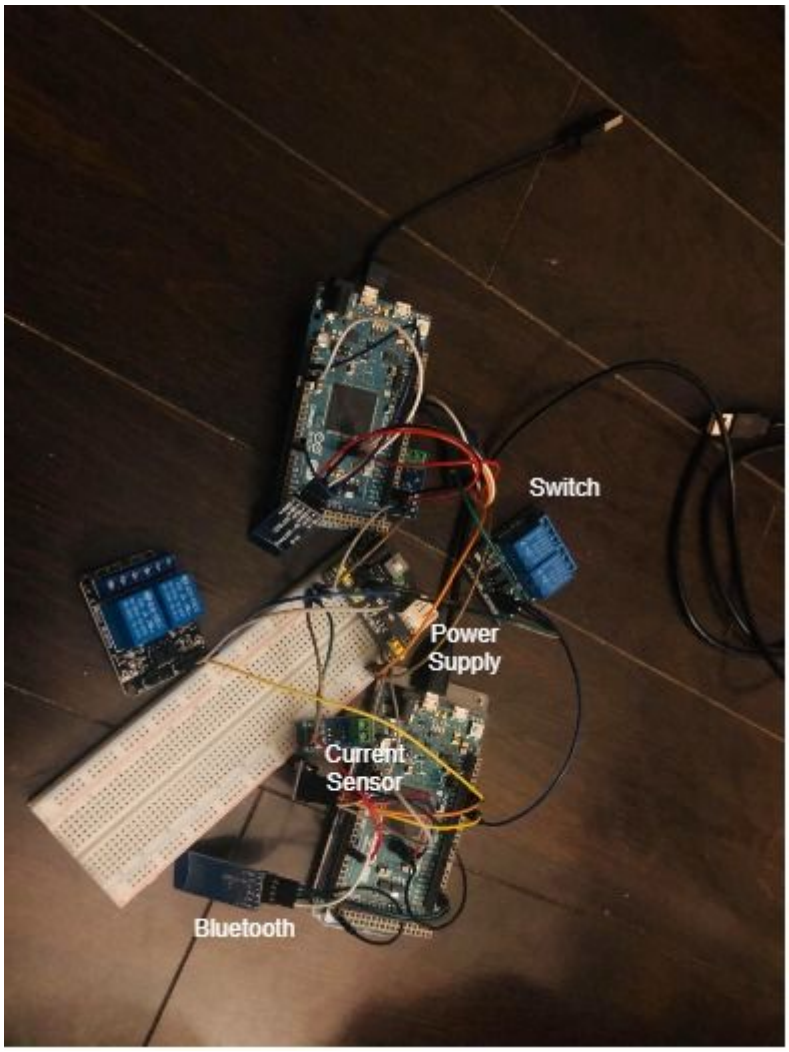


Figure 1: System level overview

MODULE LEVEL DESIGN

Arduino Due



For the Arduino Due module, there are two functionalities/modes.

1. Accept instructions from the android application via bluetooth and communicate with the Arduino Due board from the other household, obtaining real time battery level of the current household.
   Input: Bluetooth signal/instructions given by mobile application.
   Output: Arduino board from the other side
   Function: The type of transaction (buy/sell) and how much power is involved in this operation is delivered by the mobile application via bluetooth, and Arduino Due tells the converter to work (on/off) according to the instructions. At the same time, Arduino Due on both sides will tell their own Android devices about their respective battery-level via current sensor.

2. Control switch between two households
   Input: bluetooth device of Android device
   Output: connecting switch between the two households
   Function: When buy/sell energy, the android device will send a bluetooth signal to another household, applying for trade. If there is an agreement, the switch will close, and when there is no transaction happening, the switch will be off.

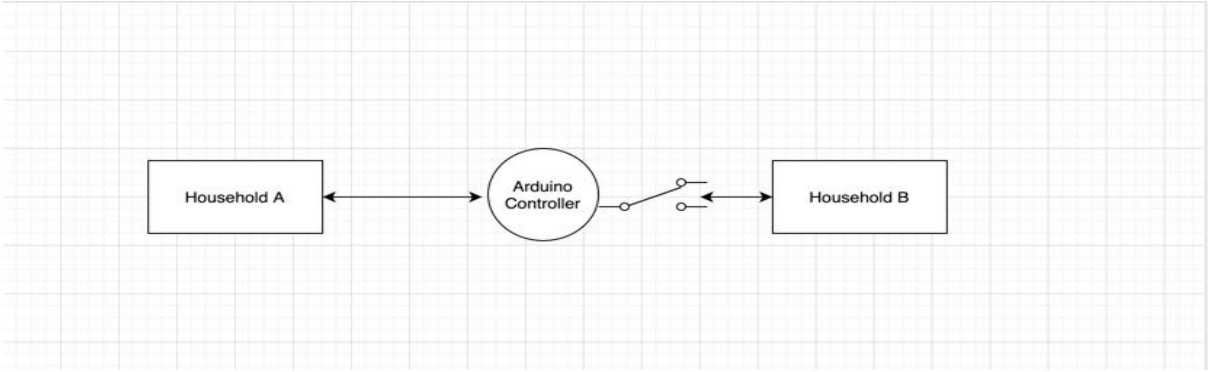


Figure 2

3. Control switches between each solar charger and its battery charged
   Input: current sensor
   Output: connecting switches between each solar charger and its household
   Function: Current sensor will get the information of the battery-level of each battery, giving back to the Arduino board, then if the battery-level is overloaded, switch doesn't work, versus vice.

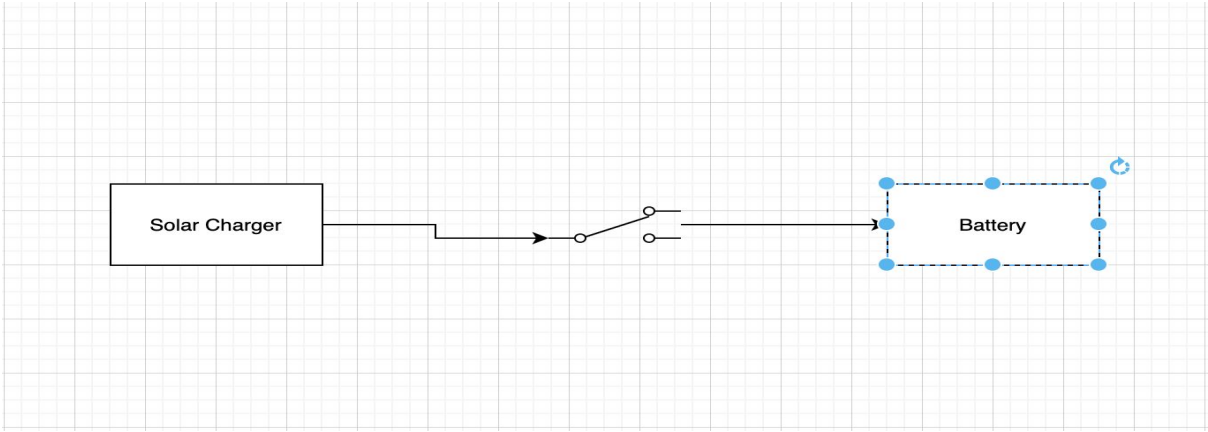


Figure 3

Mobile Application

- Communication with the Arduino bluetooth module
  Input: Progress of energy transfer

Output: Type of command (send/receive energy)
Description: The only device communicating with the mobile application is the Arduino bluetooth module. When the user initiates an energy transfer, the device will notify the Arduino via a bluetooth connection. The notification will contain the details of the action. Data being sent from the Arduino contains the progress of the energy transfer, and will be used to update the application progress bar in real time.

1. **Auto Energy Transfer Mode**: This mode will configure the app to automatically transfer energy to maintain a preset energy level threshold.

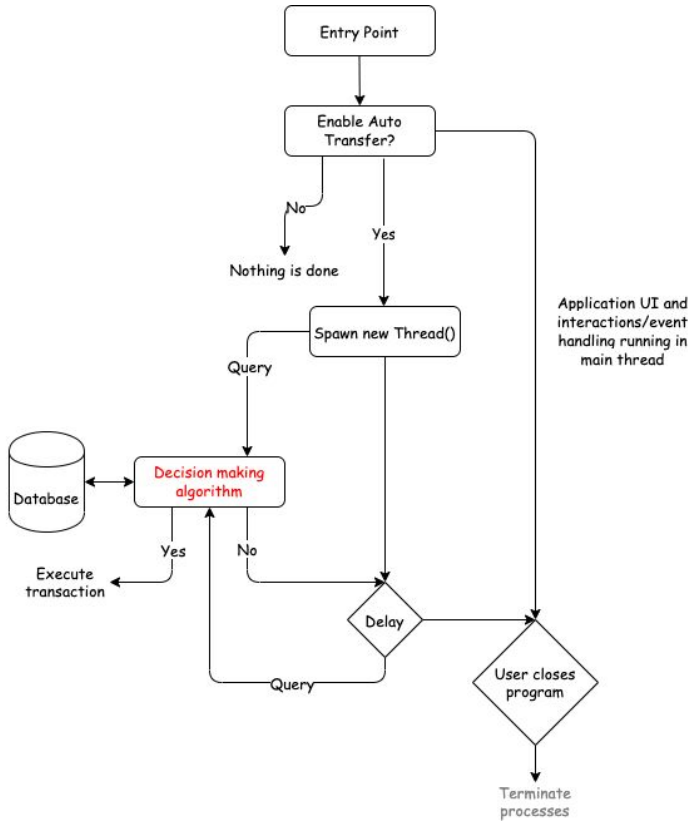


Figure 4a: Application pipeline

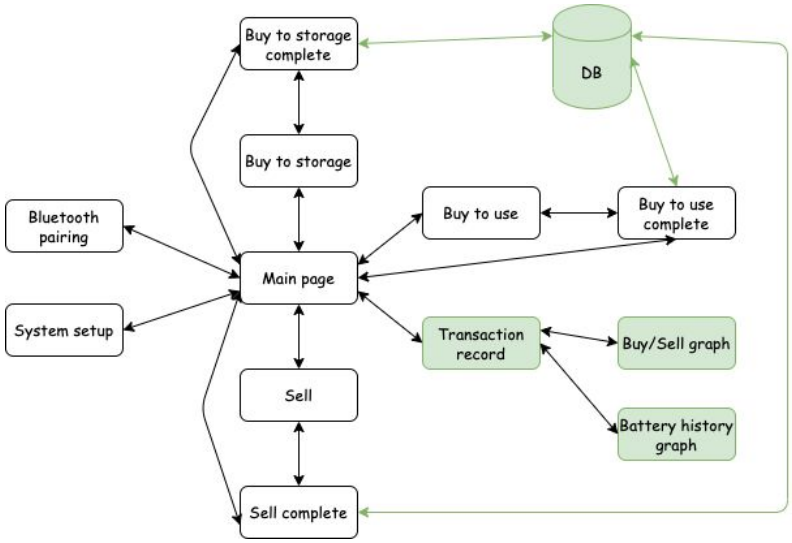


Figure 4b: Main thread UI flowchart, newly added components and interactions are in green

2. **Battery Level and Transaction History**: The new design introduces an interface that allows the user to check battery level history and transaction history, as well as to see a graphical representation of the history.

The way the newer version of the application communicates with the rest of the system is by tweaking and reusing the existing protocol which has been established in the old version.

The new features revolve around the implementation of a local database, which will store all the user data including battery levels and transaction history. Anytime a transaction is executed, whether it is buying (manual or auto), or selling, the transaction data will be stored in the history table. The system requests battery information from the controller periodically, and stores the data in the battery table. The battery level obtained from every request is compared to a preset ideal battery level, and if it is below the ideal level, the system will make an attempt to compensate the difference by initiating an auto-buy transaction.

Our two main tables are:

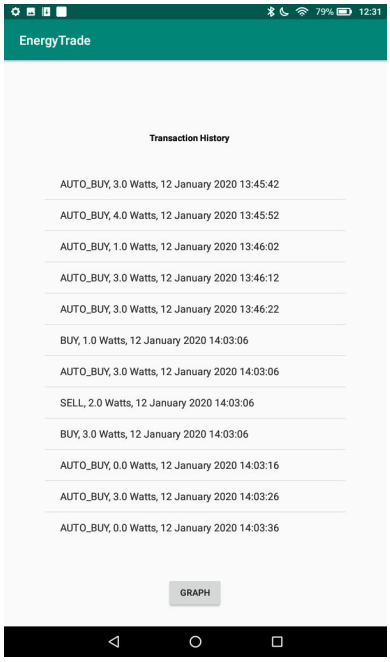
@Table History

id	type	time	amount
text/UUID	**text/String** Ex. “AUTO_BUY”, “SELL”	**text/Date**	**real/double**

@Table Battery

id	level	time
text/UUID	**real/double**	**text/Date**

We have added two new activities, which looks like the following:



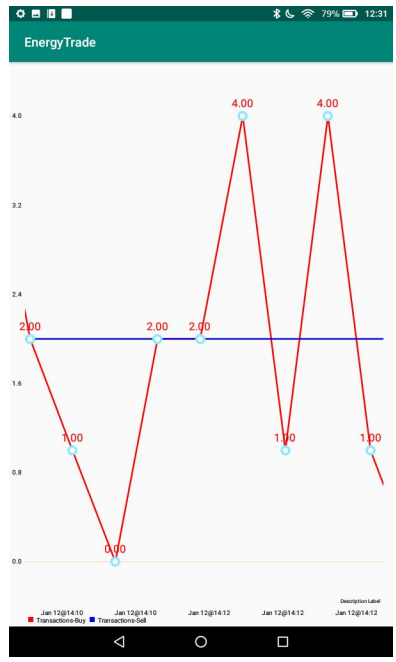


Figure 5 (left) and Figure 6 (right)

The first activity displays the list of past transactions, in this case they are generated by our test code. The second activity then displays a line chart representation of the transaction history. The two new activities can be directly accessed from the main page of the application.

Electrical System

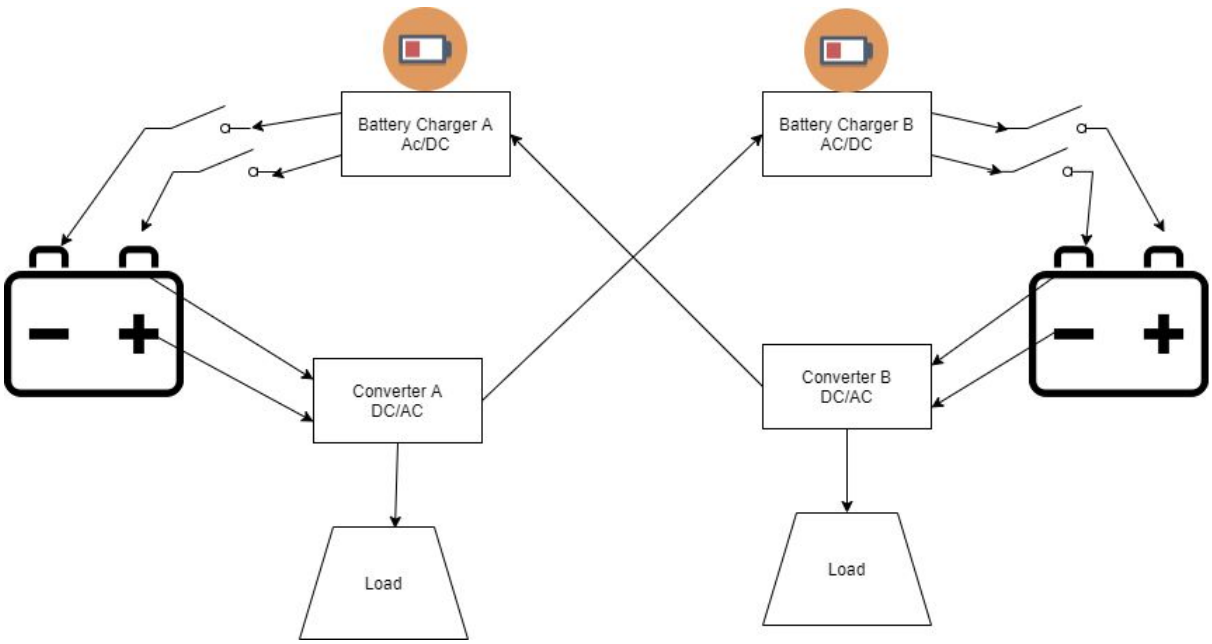


Figure 7

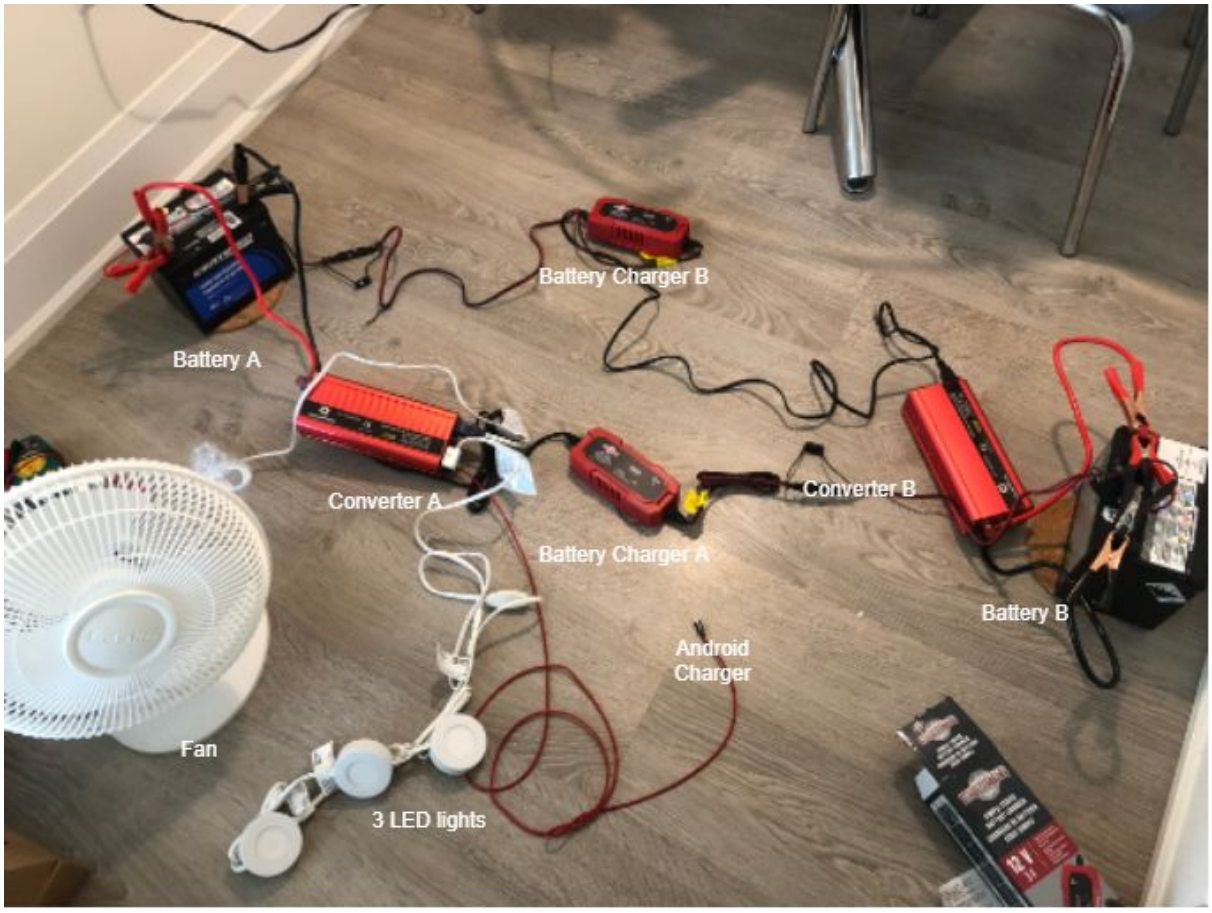


Figure 8

DC-AC Converter

- Provides energy to the load and connects the separate systems into a grid
  Input: 12 V DC battery as the power source
  Output: 120 V AC Power supply to the loads and to the battery charger
  Description: This converts DC supply of the battery to AC supply that is used as the power supply to the load which include a fan, 3 led lights and an android charger as well as provides supply to the connection for the battery charger for the grid.

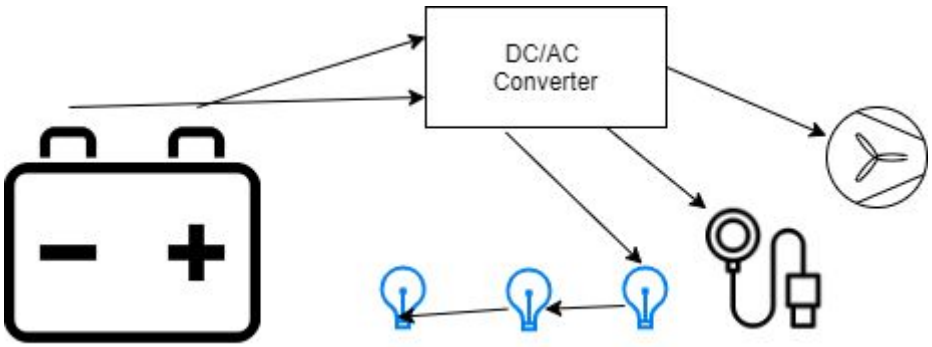


Figure 9

AC-DC Battery Charger

- Provides energy to the grid so it charge other units
  Input: 120 V AC Power supply
  Output: 12 V 3 A DC Power supply for the battery
  Description: This converts AC supply in the system into DC supply which is used to provide power to the battery.

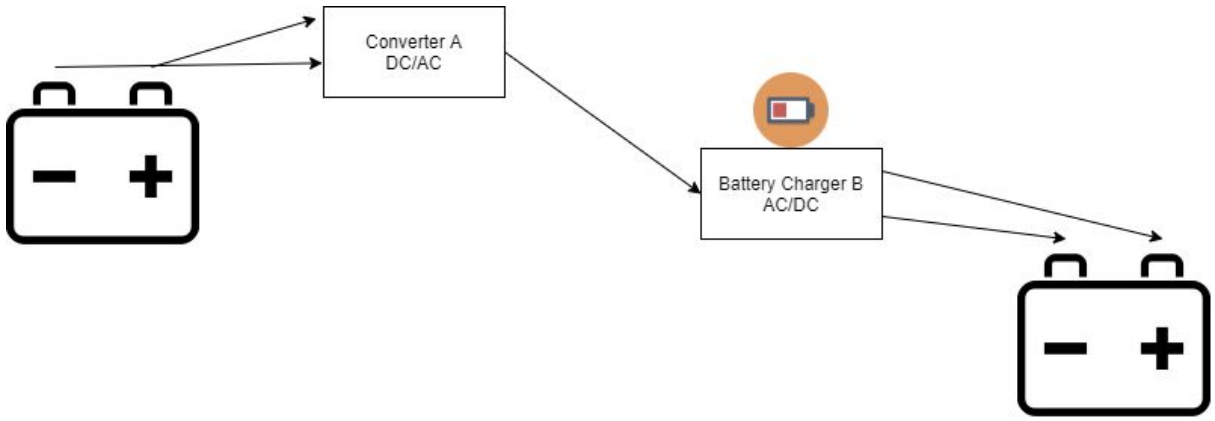


Figure 10

ASSESSMENT OF THE DESIGN

Arduino Controller
The strength is that the Arduino board is a very flexible DIY tool, that you can basically use it as a bridge which can create communication between converters, batteries and Android devices. However, the weakness is that there are so many tiny modules connected to this board, if one of the modules doesn't work, it will cause problems for the whole system. After we discussed, we decided to proceed with the hardwell assembling part first and test/finish each module one by one.

Mobile Application
The new version of the mobile application adds some minor features on top of the old design, making it a more stateful application which holds user data. The new application also incorporates some more robustness and better exception handling with refactored code. Furthermore, it automates the transaction process, which improves the overall user experience.

DC-AC Converter
In this we needed to provide AC power to the user so that they can use normal AC components that are universally used. We decided to use a 12V DC-AC converter as we felt that it was the best solution instead of using a DC-DC converter as it would require more

complex design for the user to use. This allows the user to use their load which can consist of a fan, 3 led lights and a charger simultaneously.

AC-DC Battery Charger

This is used to charge a battery using an AC source. In the grid there is power in AC that we can use to charge the battery. Essentially this allows the users to trade energy and send energy to the batteries.

Testing and Verification

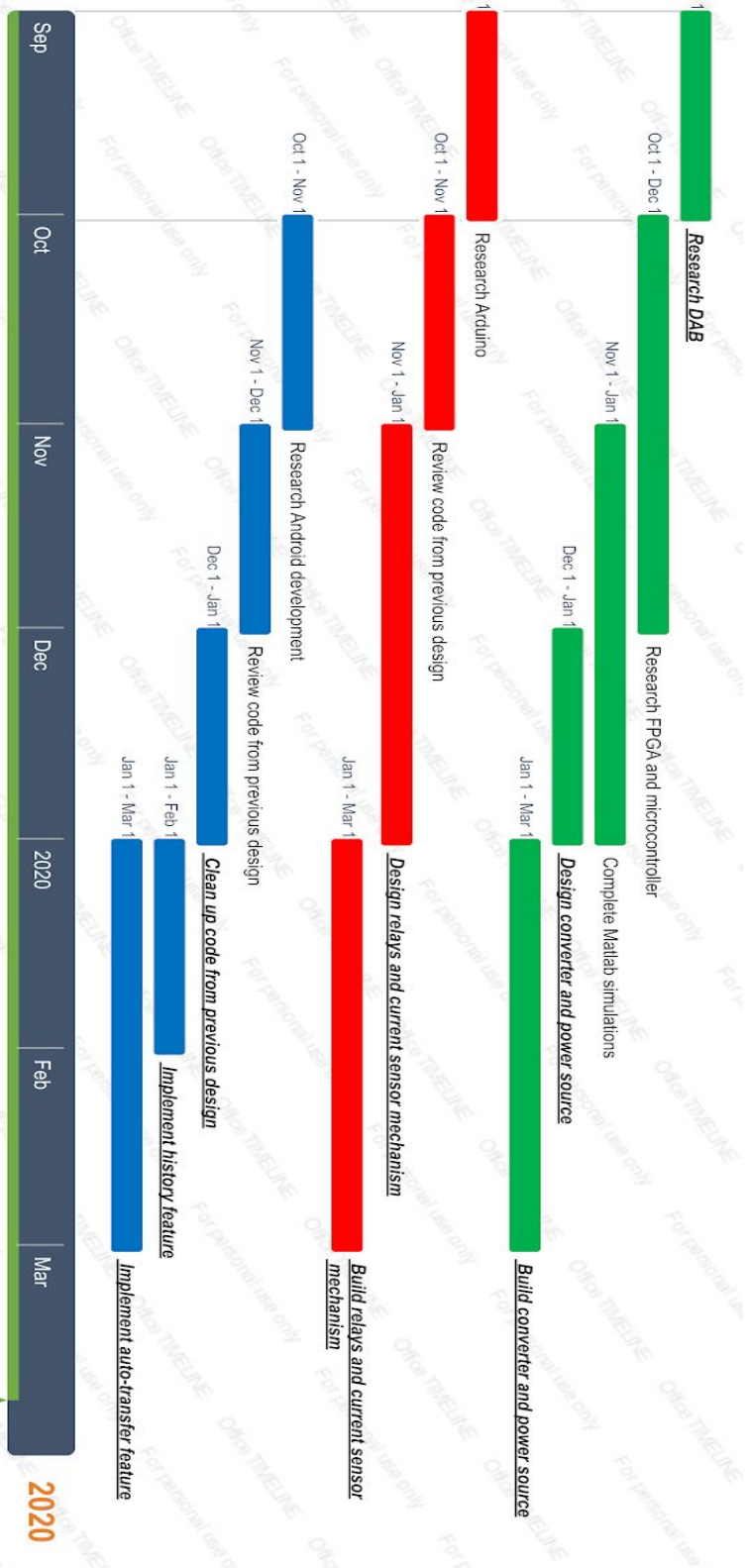
Verification Table

ID	Requirement	Verification Method	Verification Result and Proof
1	Bluetooth works well	Test: send a signal from Android device to Arduino, there will be log shown if connected	Pass
2.1	Switches off when battery is fully charged	Test: DC meter connected to the battery and see if switches off after fully charged	Pass
2.2	Switches work between households	Test: send a signal from Android device and see if switches on/off	Pass
3	Getting real time battery level	Test: see the value of battery level on the Android device, graphing	Pass
4	Recording data after transactions in the android app	Test: Calling the underlying transaction method and test if a new entry is written to the database	Pass
5	Sync chart with database updates in the android app	Test: Checking whether the chart holds the new data when it is written to the database	Pass

6	**Average load for long period of time**	**Test: Apply load of an android charger and 2 led lights for 8 hours**	**Pass**
7	**Maximum load**	**Test: Apply maximum load of a fan, 3 led lights and 3 chargers for 1 hour**	**Pass**
8	**Battery Charging**	**Test: Connect the system to allow Battery B to charge Battery A from 50% to 100%**	**Pass**

WORK PLAN

GANTT CHART AND WORK BREAKDOWN



FINANCIAL PLAN

Items	Quantity	Price	Total
DC-AC Converter	2	$90.00	$180.00
Battery Charger	2	$65.00	$130.00
12V Battery	2	$110.00	$220.00
Amp meter	1	$40.00	$40.00
LED lights	1	$20.00	$20.00
Fan	1	$30.00	$30.00
wires	1	$20.00	$20.00
Android device	2	$75.00	$150.00
Arduino board	1	$75.00	$75.00
Arduino parts (current sensor, bluetooth)	2	$45.00	$90.00
breadboard	1	$15.00	$15.00
total			$970.00

FEASIBILITY ASSESSMENT

- Key skills/knowledge:
  - Software engineering (mobile) using Java/Kotlin
  - Arduino & embedded software programming to control power lines
  - Analog circuit design skills
  - Workings of AC and DC power
  - Details about workings of converter and battery chargers
- Key resources:
  - Arduino hardware components
  - Analog circuit components

    - Android testing machines
    - DC/AC converters
    - AC/DC battery chargers
- Risk assessment:
    - Potential risk, network problems: failing to establish a bluetooth connection between device and all of the other nodes. In this case, it would be better to allow the nodes to communicate by physical wiring. Downscaling the performance would not be helpful in solving the problem in this case.
    - Potential risk with handling power in the system and with the connections with the batteries.

CONCLUSION

The system receives through 12V batteries and provides AC power to the user. It allows the user to communicate with the system using a cell phone app that is connected through bluetooth. This allows the user to see how much Energy they currently have stored, so that they can either use the energy, sell the energy if there is a surplus or get the system to get more energy by either producing more or buying from another unit. We feel that this would be a good fit for an environment such as a refugee camp that would allow their energy requirements to be met while being a system that is relatively inexpensive, durable, portable and easy to operate for the users.

REFERENCES

[1] “What is a Refugee Camp? Definition and Statistics: USA for UNHCR,” *Definition and Statistics | USA for UNHCR*. [Online]. Available: https://www.unrefugees.org/refugee-facts/camps/. [Accessed: 18-Sep-2019].

[2] “World's Largest Refugee Camps in 2018,” *raptim*. [Online]. Available: https://www.raptim.org/largest-refugee-camps-in-2018/. [Accessed: 18-Sep-2019].

[3] United Nations, “Climate change and disaster displacement,” *UNHCR*. [Online]. Available: https://www.unhcr.org/climate-change-and-disasters.html. [Accessed: 18-Sep-2019].

[4] E. C. Dunn, “The Failure of Refugee Camps,” *Boston Review*, 24-Nov-2015. [Online]. Available: http://bostonreview.net/editors-picks-world/elizabeth-dunn-failure-refugee-camps. [Accessed: 19-Sep-2019].

[5] "ISO 13823:2008", *ISO*, 2019. [Online]. Available: https://www.iso.org/standard/39021.html. [Accessed: 19- Sep- 2019].

[6] Ergonomics of human-system interaction - Usability methods supporting human-centred design, ISO/TR 16982:2002, 06/01/2002

[7] Piyush G. Chapla. (2004, Aug.). “HOW MUCH WEIGHT IS TOO MUCH FOR MANUAL LIFTING: DETERMINING A WEIGHT LIMIT GUIDELINE FOR TEAM-EFFORT LIFTING TASKS,” *New Jersey’s Science and Technology University* [Online]. http://archives.njit.edu/vol01/etd/2000s/2004/njit-etd2004-111/njit-etd2004-111.pdf [Accessed Sep. 18, 2019].

[8] Health and Safety Executive (2004). *Getting to grips with manual handling* [Online]. Available: http://www.healthandsafetyworksni.gov.uk/getting_2_grips.pdf [Accessed Sep. 18, 2019].

[9]*Webstore.iec.ch*, 2019. [Online]. Available: https://webstore.iec.ch/preview/info_iec62109-1%7Bed1.0%7Den.pdf. [Accessed: 19-Sep- 2019].

[10]S. Mandelli, "Effect of load profile uncertainty on the optimum sizing of off-grid PV systems for rural electrification," *Sustainable Energy Technologies and Assessments,* vol. 18, pp. 34-47, 2016.

[11] “Why Big Solar is a Colossally Bad Idea (10 Reasons Decentralized Solar is Much Better),” *CleanTechnica*, 07-May-2011. [Online]. Available: https://cleantechnica.com/2011/04/27/why-big-solar-is-a-colossally-bad-idea-10-reasons-decentralized-solar-is-much-better/. [Accessed: 18-Sep-2019].

[12] “LED Bulbs,” *The Home Depot Canada*. [Online]. Available: https://www.homedepot.ca/en/home/categories/lighting-and-ceiling-fans/light-bulbs/led-bulbs.html. [Accessed: 10-Oct-2019].

[13] *walmart.ca*. [Online]. Available: https://www.walmart.ca/en/electronics/cell-phones/cellphone-accessories/chargers/N-3440. [Accessed: 10-Oct-2019].

[14] “Household Fans: Home & Kitchen: Specialty Fans, Personal Fans, Floor Fans, Table Fans, Drum Fans & More,” *Amazon.ca: Household Fans: Home & Kitchen: Specialty Fans, Personal Fans, Floor Fans, Table Fans, Drum Fans & More*. [Online]. Available: https://www.amazon.ca/b?ie=UTF8&node=6646978011. [Accessed: 10-Oct-2019].

[15] “Arduino Official Store: Boards Shields Kits Accessories,” *Arduino Official Store | Boards Shields Kits Accessories*. [Online]. Available: https://store.arduino.cc/usa/. [Accessed: 02-Dec-2019].

[16] T. Technologies, “DE Series Introduction,” *Terasic*. [Online]. Available: https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=163#Category165. [Accessed: 03-Dec-2019].

[17] YouTube. (2019). *Android Studio For Beginners Part 1*. [online] Available at: https://www.youtube.com/watch?time_continue=1&v=dFlPARW5IX8 [Accessed 19 Sep. 2019].

[18] Arduino.cc. (2019). *Arduino - Tutorials*. [online] Available at: https://www.arduino.cc/en/Tutorial/HomePage?from=Main.Tutorials [Accessed 19 Sep. 2019].

APPENDIX C: Final Report Document Attribution Table

Section	***Student Initials***		
	1.RK	*2.KY*	*3.WY*
Executive Summary		*RS,RD,MR, ET*	
Background and Motivation	*RS,RD,MR,ET*		
Group Highlights and Individual Contributions	*RS,RD*	*RS,RD*	*RS,RD,MR,E*
Acknowledgments			*RS,RD,MR,E*
Project goals			*RS,RD,MR,ET*
Project Requirements			*RS,RD,MR,ET*
System Level Overview	*RS,RD,MR,ET*		
System block diagram	*RS,RD,MR,ET*		
Module Level Overview	*RS,RD*	*RS,RD*	*RS,RD*
Assessment of Final Design	*RS,RD*	*RS,RD*	*RS,RD*
Testing and Verification	*RS,RD*	*RS,RD*	*RS,RD*
Work breakdown structure and Gantt chart		*RD*	

Financial plan	*RS,RD,MR,ET*		
All	*CM, FP*		

Abbreviation Codes:
Fill in abbreviations for roles for each of the required content elements. You do not have to fill in every cell. The "**All**" row refers to the complete document and should indicate who was responsible for the final compilation and final read through of the completed document.

RS – responsible for research of information
RD – wrote the first draft
MR – responsible for major revision
ET – edited for grammar, spelling, and expression
OR – other
"All" row abbreviations:
FP – final read through of complete document for flow and consistency
CM – responsible for compiling the elements into the complete document
OR - other
If you put OR (other) in a cell please put it in as OR1, OR2, etc. Explain briefly below the role referred to:
OR1: enter brief description here
OR2: enter brief description here

Signatures
By signing below, you verify that you have read the attribution table and agree that it accurately reflects your contribution to this document.

Name	***RAFAY KHAN***	***Signature***	***R.K***	***Date:***	***31 March 2020***
Name	KaiWen Ying	***Signature***	***K.Y***	***Date:***	***March 31 2020***
Name	Wenxuan You	***Signature***	***W.Y***	***Date:***	***Mar.31 2020***

Appendix D: Android Application code

```
package com.capstone.energytrade;


import android.bluetooth.BluetoothSocket;


import java.io.File;

import java.io.IOException;

import java.io.InputStream;

import java.util.ArrayList;

import java.util.Collections;

import java.util.Comparator;

import java.util.Objects;


class auto_transfer {


  static Thread autoTransferThread(File DBPath, long delay, boolean enable,

                                   BluetoothSocket btSocket) {

    return new Thread(

      new Runnable() {

        @Override

        public void run() {

          while (true) {

            autoTransferListen(DBPath, delay, enable, btSocket);

          }

        }

      }

    );

  }
```

```
/* listens to the auto transfer service */

private static void autoTransferListen(File DBPath, long delay, boolean enable,

                                BluetoothSocket btSocket) {



   Double currentBatteryLevel = getCurrentBatteryLevel(btSocket);

   if (currentBatteryLevel != null) {

      DB.SaveBatteryLevel(DBPath, currentBatteryLevel);



      if (enable) {

         transferObj result = analyze(DBPath, currentBatteryLevel);



         if (result != null) {

            boolean success = transfer(result, btSocket);

            if (success) {

               DB.SaveHistory(DBPath, result.type, result.amount);

            } else {

               System.out.println("Transfer Failed");

            }

         }

      }

   }



   /* delay the interval */

   try {

      Thread.sleep(delay);

   } catch (InterruptedException ex) {

      ex.printStackTrace();

   }

}
```

```
/* get the current battery level */

private static Double getCurrentBatteryLevel(BluetoothSocket btSocket) {

   if (btSocket != null) {

      /* set 10 second timeout */

      long start = System.currentTimeMillis();

      long end = start + 10000;

      while (System.currentTimeMillis() < end) {

         try {

            btSocket.getOutputStream().write('7');

            btSocket.getOutputStream().write('\n');

            InputStream arduinoOutput = btSocket.getInputStream();

            int availability = arduinoOutput.available();

            if (availability > 0) {

               byte[] packet = new byte[availability];

               arduinoOutput.read(packet);

               String[] soc = new String(packet).split("/");

               return (double) Math.round(Double.parseDouble(soc[1].trim()) * 100) / 100;

            }

         } catch (IOException ex) {

            ex.printStackTrace();

            System.out.println("Bluetooth Connection Failed in Auto Transfer Thread");

         }

      }

   }

   return null;

}



/* analyzes the history and determine whether a transfer is needed,
```

```
 * returns transfer objects if yes, otherwise returns null */
private static transferObj analyze(File DBPath, double batteryLevel) {
    if (AutoTransferConfig.autoTransferThreshold > batteryLevel) {
        // if battery level is below our threshold, we need to perform a transfer
        return new transferObj("BUY", AutoTransferConfig.autoTransferThreshold - batteryLevel);
    }
    return null;
}

/* performs the transfer by communicating with arduino */
private static boolean transfer(transferObj transferInfo, BluetoothSocket btSocket) {
    if (transferInfo == null || btSocket == null) return false;

    if (transferInfo.type == "BUY") {
        byte[] buffer = new byte[1024];
        int bufferIndex = 0;
        try {
            btSocket.getOutputStream().write('1');
            btSocket.getOutputStream().write('\n');
            InputStream arduinoOutput = btSocket.getInputStream();
            int available = arduinoOutput.available();
            ArrayList<Offer> offerList = new ArrayList<>();
            if (available > 0) {
                byte[] packet = new byte[available];
                arduinoOutput.read(packet);
                for (int i = 0; i < available; i++) {
                    byte b = packet[i];
                    if (b != 0) {
                        buffer[bufferIndex++] = b;
                    }
```

```
    }
    byte[] encoded = new byte[bufferIndex];
    System.arraycopy(buffer, 0, encoded, 0, encoded.length);
    String[] offers = new String(encoded).split("\n");
    for (int i = 0; i < offers.length; i++) {
        String[] offerDetails = offers[i].split("/");
        if (offerDetails.length >= 6) {
            offerList.add(new Offer(offers[0], offers[1], offers[2], offers[3],
                offers[4], offers[5]));
        }
    }
}
ArrayList<Offer> potentialOffers = new ArrayList<>();
for (Offer offer : offerList) {
    if (Double.parseDouble(offer.quantity) < transferInfo.amount)
        potentialOffers.add(offer);
}
while (potentialOffers.size() > 0 && transferInfo.amount > 0) {
    Offer bestOffer = Collections.min(potentialOffers, new Comparator<Offer>() {
        @Override
        public int compare(Offer a, Offer b) {
            Double a_ = Double.parseDouble(a.price);
            Double b_ = Double.parseDouble(b.price);
            return a_.compareTo(b_);
        }
    });
    btSocket.getOutputStream().write('2');
    btSocket.getOutputStream().write('/');
    if (Integer.parseInt(bestOffer.nodeID) < 10) {
        btSocket.getOutputStream().write('0');
```

```
                btSocket.getOutputStream().write(bestOffer.nodeID.getBytes());
            } else {
                btSocket.getOutputStream().write(bestOffer.nodeID.getBytes());
            }
            btSocket.getOutputStream().write('/');
            btSocket.getOutputStream().write(bestOffer.price.getBytes());
            btSocket.getOutputStream().write('/');
            btSocket.getOutputStream().write(bestOffer.quantity.getBytes());
            btSocket.getOutputStream().write('/');
            btSocket.getOutputStream().write(bestOffer.count.getBytes());
            btSocket.getOutputStream().write('/');
            btSocket.getOutputStream().write(bestOffer.path.getBytes());
            btSocket.getOutputStream().write('\n');
            transferInfo.amount -= Double.parseDouble(bestOffer.quantity)/AutoTransferConfig.batteryCapacity;
            potentialOffers.removeIf(o -> (o.equals(bestOffer)));
          }
        } catch (IOException | NumberFormatException ex) {
          ex.printStackTrace();
          System.out.println("Bluetooth Connection Failed in Auto Transfer Thread");
        }
      }
      return true;
    }
}


/* specifies the type and amount of a transfer */
class transferObj {
    final String type;
    double amount;
```

```
    public transferObj(String type, double amount) {
        this.type = type;
        this.amount = amount;
    }
}

class Offer {
    String signal;
    String nodeID;
    String price;
    String quantity;
    String count;
    String path;

    public Offer(String signal, String nodeID, String price, String quantity, String count,
                 String path) {
        this.signal = signal;
        this.nodeID = nodeID;
        this.price = price;
        this.quantity = quantity;
        this.count = count;
        this.path = path;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Offer offer = (Offer) o;
        return Objects.equals(signal, offer.signal) &&
```

```
        Objects.equals(nodeID, offer.nodeID) &&
        Objects.equals(price, offer.price) &&
        Objects.equals(quantity, offer.quantity) &&
        Objects.equals(count, offer.count) &&
        Objects.equals(path, offer.path);
    }

    @Override
    public int hashCode() {
        return Objects.hash(signal, nodeID, price, quantity, count, path);
    }
}

class AutoTransferConfig {
    static final Double batteryCapacity = 100.0;
    static final Double autoTransferThreshold = 0.5;
}

package com.capstone.energytrade;

import android.app.Application;
import android.bluetooth.BluetoothAdapter;
import android.bluetooth.BluetoothDevice;
import android.bluetooth.BluetoothSocket;
import android.widget.Toast;

import java.io.IOException;
import java.util.UUID;
```

```
public class BluetoothApplication extends Application {


    static final UUID myUUID = UUID.fromString("00001101-0000-1000-8000-00805F9B34FB");

    private static BluetoothApplication sInstance;

    BluetoothAdapter myBluetooth = null;

    BluetoothSocket btSocket = null;

    private boolean isBtConnected = false;

    private String address_setup = null;

    private boolean ConnectSuccess = true;


    public static BluetoothApplication getApplication() {

        return sInstance;

    }


    public void onCreate() {

        super.onCreate();


        sInstance = this;

    }


    public void setupBluetoothConnection(String address_setup) {

        // Either setup your connection here, or pass it in


        try {

            if (btSocket == null || !isBtConnected) {

                myBluetooth = BluetoothAdapter.getDefaultAdapter();//get the mobile bluetooth device

                BluetoothDevice dispositivo = myBluetooth.getRemoteDevice(address_setup);

                //connects to

                // the device's address and checks if it's available

                btSocket = dispositivo.createInsecureRfcommSocketToServiceRecord(myUUID);//create a
```

```
            // RFCOMM (SPP) connection
            BluetoothAdapter.getDefaultAdapter().cancelDiscovery();
            btSocket.connect();//start connection

          }
        } catch (IOException e) {
          ConnectSuccess = false;//if the try failed, you can check the exception here
          msg("Bluetooth Connection Failed.");
//              Intent failtoconnect = new Intent (this, btPairing.class);
//              startActivity(failtoconnect);
        }
    }

    public BluetoothSocket getCurrentBluetoothConnection() {
        return btSocket;
    }

    private void msg(String s) {
        Toast.makeText(getApplicationContext(), s, Toast.LENGTH_LONG).show();
    }
}

package com.capstone.energytrade;

import android.bluetooth.BluetoothAdapter;
import android.bluetooth.BluetoothDevice;
import android.content.Intent;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.view.View;
```

```
import android.widget.AdapterView;

import android.widget.AdapterView.OnItemClickListener;

import android.widget.ArrayAdapter;

import android.widget.Button;

import android.widget.ListView;

import android.widget.TextView;

import android.widget.Toast;


import java.util.ArrayList;

import java.util.Set;


import static com.capstone.energytrade.DB.DBName;


public class btPairing extends AppCompatActivity {


   Button btnPaired;

   ListView devicelist;

   private Set<BluetoothDevice> pairedDevices;

   private BluetoothAdapter myBluetooth = null;

   private OnItemClickListener myListClickListener = new OnItemClickListener() {

      public void onItemClick(AdapterView av, View v, int arg2, long arg3) {

         // Get the device MAC address, the last 17 chars in the View

         String info = ((TextView) v).getText().toString();

         String address = info.substring(info.length() - 17);

         // Make an intent to start next activity.

         Intent i = new Intent(btPairing.this, setup_interface.class); /* Testing the transaction

    history page*/

         //Change the activity.

         BluetoothApplication.getApplication().setupBluetoothConnection(address);

//          i.putExtra(EXTRA_ADDRESS, address);//this will be received at ledControl (class)
```

```
//          Activity
            startActivity(i);
        }
    };

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_bt_pairing);

        /* Database Test! Delete Afterwards */
        DB.DeleteDB(this.getDatabasePath(DBName));
        String status = DB.SaveHistory(this.getDatabasePath(DBName), "BUY", 1.0);
        DB.SaveHistory(this.getDatabasePath(DBName), "SELL", 2.0);
        Toast.makeText(getApplicationContext(), status, Toast.LENGTH_LONG).show();
        /*-----------------------------------*/

        btnPaired = findViewById(R.id.button);
        devicelist = findViewById(R.id.listview);
        myBluetooth = BluetoothAdapter.getDefaultAdapter();
        if (myBluetooth == null) {
            //Show a mensag. that the device has no bluetooth adapter
            Toast.makeText(getApplicationContext(), "Bluetooth Device Not Available",
                Toast.LENGTH_LONG).show();
            //finish apk
            finish();
        } else {
            if (myBluetooth.isEnabled()) {
                //System.out.print("bluetooth");
            } else {
```

```
            //Ask to the user turn the bluetooth on
            Intent turnBTon = new Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE);
            startActivityForResult(turnBTon, 1);
        }
    }

    btnPaired.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            pairedDevicesList(); //method that will be called
        }
    });
}

private void pairedDevicesList() {
    pairedDevices = myBluetooth.getBondedDevices();
    ArrayList list = new ArrayList();

    if (pairedDevices.size() > 0) {
        for (BluetoothDevice bt : pairedDevices) {
            list.add(bt.getName() + "\n" + bt.getAddress()); //Get the device's name and the
           // address
        }
    } else {
        Toast.makeText(getApplicationContext(), "No Paired Bluetooth Devices Found.",
            Toast.LENGTH_LONG).show();
    }

    final ArrayAdapter adapter = new ArrayAdapter(this, android.R.layout.simple_list_item_1, list);
    devicelist.setAdapter(adapter);
```

```
        devicelist.setOnItemClickListener(myListClickListener); //Method called when the device from
        // the list is clicked
    }
}
package com.capstone.energytrade;


import android.graphics.Color;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;


import com.github.mikephil.charting.charts.LineChart;
import com.github.mikephil.charting.components.XAxis;
import com.github.mikephil.charting.components.YAxis;
import com.github.mikephil.charting.data.Entry;
import com.github.mikephil.charting.data.LineData;
import com.github.mikephil.charting.data.LineDataSet;
import com.github.mikephil.charting.formatter.ValueFormatter;
import com.github.mikephil.charting.interfaces.datasets.ILineDataSet;


import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;


/*
 * STATUS: LOOKS OK
 * */


public class battery_level_graph extends AppCompatActivity {


    List<DataPoint> battery_data;
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_battery_level_graph);
    loadData();
    render();
}

/* loads the data from database */
private void loadData() {
    ArrayList<DB.BatteryContent> battery_contents =
        DB.GetBatteryContent(this.getDatabasePath(DB.DBName), "select * from battery");
    battery_data =
        battery_contents.stream().map(d -> new DataPoint(d.getTime(), d.getLevel())).collect(Collectors.toList());
}

/* render the graph */
private void render() {
    LineChart chart = findViewById(R.id.chart1);

    /* format axis */
    XAxis xAxis = chart.getXAxis();
    YAxis leftAxis = chart.getAxisLeft();
    YAxis rightAxis = chart.getAxisRight();

    /* customize xaxis */
    xAxis.setPosition(XAxis.XAxisPosition.BOTTOM);
    xAxis.setDrawGridLines(false);
    xAxis.setDrawAxisLine(false);
```

```
xAxis.setValueFormatter(new ValueFormatter() {
    @Override
    public String getFormattedValue(float value) {
        return conversion.timestampToDate(value);
    }
});

/* customize yaxis */
leftAxis.setDrawGridLines(false);
leftAxis.setDrawAxisLine(false);
rightAxis.setEnabled(false);

List<Entry> battery_entries = new ArrayList<>();
for (DataPoint dp : battery_data) {
    float timestamp = conversion.dateToTimestamp(dp.date);
    if (timestamp != -1) {
        battery_entries.add(new Entry(conversion.dateToTimestamp(dp.date),
            (float) dp.amount));
    }
}
LineDataSet battery_dataSet = new LineDataSet(battery_entries, "Battery-Level");
battery_dataSet.setAxisDependency(YAxis.AxisDependency.LEFT);

battery_dataSet.setColor(Color.RED);
battery_dataSet.setValueTextColor(Color.RED);

List<ILineDataSet> dataSets = new ArrayList<>();
dataSets.add(battery_dataSet);
LineData lineData = new LineData(dataSets);
chart.setData(lineData);
```

```
        chart.invalidate();
    }



    /* object representing a data point */
    class DataPoint {
        String date;
        double amount;

        public DataPoint(String date, double amount) {
            this.date = date;
            this.amount = amount;
        }
    }
}

package com.capstone.energytrade;

import android.bluetooth.BluetoothSocket;
import android.content.Intent;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.view.View;
import android.widget.Button;
import android.widget.ProgressBar;
import android.widget.TextView;
import android.widget.Toast;

import java.io.IOException;
import java.io.InputStream;
```

```
import java.util.Timer;

import java.util.TimerTask;


public class buytobattery_complete extends AppCompatActivity {


    private ProgressBar progressbar_btbc;

    private Button CT_btbc;

    private Button returntomain_btbc;

    private TextView text1_btbc;

    private InputStream inStream_btbc = null;


    private BluetoothSocket btSocket_btbc =

        BluetoothApplication.getApplication().getCurrentBluetoothConnection();


    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_buytobattery_complete);


        progressbar_btbc = findViewById(R.id.progressbar_btbc);

        CT_btbc = findViewById(R.id.CT_btbc);

        returntomain_btbc = findViewById(R.id.returntomain_btbc);

        text1_btbc = findViewById(R.id.text1_btbc);



        progressCheck();


        CT_btbc.setOnClickListener(new View.OnClickListener() {

            @Override

            public void onClick(View v) {
```

```
            if (btSocket_btbc != null) {
                try {
                    btSocket_btbc.getOutputStream().write('5');
                    btSocket_btbc.getOutputStream().write('\n');
                } catch (IOException e) {
                    msg("Error");
                }


            }
            system_status.btbStatus = true;
            finish();
        }

    });


    returntomain_btbc.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            Intent returntomain_btbc = new Intent(buytobattery_complete.this,
                setup_interface.class);
            startActivity(returntomain_btbc);
        }
    });


}



private void progressCheck() {
    if (btSocket_btbc != null) {
        try {
```

```
btSocket_btbc.getOutputStream().write('7');

btSocket_btbc.getOutputStream().write('\n');

Timer timer = new Timer();

timer.schedule(new TimerTask() {

    @Override

    public void run() {

        runOnUiThread(new Runnable() {

            @Override

            public void run() {

                try {

                    inStream_btbc = btSocket_btbc.getInputStream();

                    int inStreamAvailable = inStream_btbc.available();

                    if (inStreamAvailable > 0) {

                        byte[] packetBytes_btbc = new byte[inStreamAvailable];

                        inStream_btbc.read(packetBytes_btbc);

                        System.out.println(packetBytes_btbc);

                        String[] socInfo = new String(packetBytes_btbc).split("/");

                        String stateofcharge_btbc = socInfo[1].trim();

                        System.out.println(socInfo);

                        text1_btbc.setText(stateofcharge_btbc + "%");



                        progressbar_btbc.setProgress(Math.round(Float.parseFloat(stateofcharge_btbc)));

                    }

                } catch (IOException E) {

                    msg("Error");

                }

            }

        });



    }
```

```
            }, 1500);//Update text every second


        } catch (IOException e) {

            msg("Error");

        }

    }




  }


  private void msg(String s) {

    Toast.makeText(getApplicationContext(), s, Toast.LENGTH_LONG).show();

  }

}


package com.capstone.energytrade;


import android.bluetooth.BluetoothSocket;

import android.content.Intent;

import android.os.Bundle;

import android.support.v7.app.AppCompatActivity;

import android.view.View;

import android.widget.Button;

import android.widget.ImageButton;

import android.widget.TextView;

import android.widget.Toast;


import java.io.IOException;

import java.io.InputStream;
```

```
import java.util.Timer;

import java.util.TimerTask;



public class buytouse extends AppCompatActivity {




    private ImageButton return_btu;

    private TextView text1_btu;

    private Button confirm_btu;

    private InputStream inStream_btu = null;

    private BluetoothSocket btSocket_btu =

        BluetoothApplication.getApplication().getCurrentBluetoothConnection();



    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_buytouse);

        return_btu = findViewById(R.id.return_btu);

        confirm_btu = findViewById(R.id.confirm_btu);

        text1_btu = findViewById(R.id.text1_btu);




        return_btu.setOnClickListener(new View.OnClickListener() {

            @Override

            public void onClick(View v) {

                finish();

            }

        });



        confirm_btu.setOnClickListener(new View.OnClickListener() {
```

```
@Override
public void onClick(View v) {
    if (btSocket_btu != null) {
        try {
            btSocket_btu.getOutputStream().write('4');
            btSocket_btu.getOutputStream().write('\n');


            Timer timer = new Timer();
            timer.schedule(new TimerTask() {
                @Override
                public void run() {
                    runOnUiThread(new Runnable() {
                        @Override
                        public void run() {
                            try {
                                inStream_btu = btSocket_btu.getInputStream();
                                int bytesAvailable_btu = inStream_btu.available();
                                if (bytesAvailable_btu > 0) {
                                    byte[] packetBytes_btu =
                                        new byte[bytesAvailable_btu];
                                    inStream_btu.read(packetBytes_btu);
                                    String[] instruction =
                                        new String(packetBytes_btu).split("\n");
                                    if (instruction[0].equals("3")) {
                                        system_status.btuStatus = false;
                                        Intent nextpage_btu =
                                            new Intent(buytouse.this,
                                                buytouse_complete.class);
                                        startActivity(nextpage_btu);
                                    }
```

```
                    } else {
                        msg("No energy is available to buy.");
                    }

                } catch (IOException e) {

                }

            }
        });


        }
    }, 2000);


    } catch (IOException e) {
        msg("Error");
    }

  }

 }
});

}

private void msg(String s) {
    Toast.makeText(getApplicationContext(), s, Toast.LENGTH_LONG).show();
}
```

```
}

package com.capstone.energytrade;

import android.bluetooth.BluetoothSocket;
import android.content.Intent;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;

import java.io.IOException;

public class buytouse_complete extends AppCompatActivity {

    private TextView text1_btuc;
    private Button main_btuc;
    private Button cancel_btuc;
    private BluetoothSocket btSocket_btuc =
        BluetoothApplication.getApplication().getCurrentBluetoothConnection();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_buytouse_complete);

        text1_btuc = findViewById(R.id.text1_btuc);
        main_btuc = findViewById(R.id.main_btuc);
```

```
    cancel_btuc = findViewById(R.id.cancel_btuc);


    main_btuc.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            Intent rtm_btuc = new Intent(buytouse_complete.this, setup_interface.class);
            startActivity(rtm_btuc);
        }
    });


    cancel_btuc.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            if (btSocket_btuc != null) {
                try {
                    btSocket_btuc.getOutputStream().write('5');
                    btSocket_btuc.getOutputStream().write('\n');
                } catch (IOException e) {
                    msg("Error");
                }


            }
            system_status.btuStatus = true;
            finish();
        }
    });
}


private void msg(String s) {
    Toast.makeText(getApplicationContext(), s, Toast.LENGTH_LONG).show();
```

```
    }
}

package com.capstone.energytrade;

import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.Locale;

class conversion {
    static final String datePattern = "dd MMMM yyyy HH:mm:ss";
    static final String dmy = "dd MMMM yyyy";
    private static final long begin2020 = 1577836800L;

    /* quick customized datetime to pattern conversion */
    static String format(Date date, String pattern) {
        SimpleDateFormat myDateFormat = new SimpleDateFormat(pattern, Locale.ENGLISH);
        return myDateFormat.format(date);
    }

    /* quick datetime parsing */
    static Date parse(String date, String pattern) {
        SimpleDateFormat parser = new SimpleDateFormat(pattern, Locale.ENGLISH);
        try {
            return parser.parse(date);
        } catch (ParseException ex) {
            return null;
        }
    }
```

```
/* function to get current date in string format */

static String EncodeTimeString() {

   SimpleDateFormat myDateFormat = new SimpleDateFormat(datePattern, Locale.ENGLISH);

   return myDateFormat.format(new Date());

}



/* function that converts datetime string into timestamp */

/* notice that we use midnight Jan.2020 as reference because

 * return type has to be float and could overflow if using unix timestamp */

static float dateToTimestamp(String date) {

   SimpleDateFormat formatter = new SimpleDateFormat(datePattern, Locale.ENGLISH);

   Date dateTime;

   try {

      dateTime = formatter.parse(date);

   } catch (ParseException ex) {

      return -1;

   }

   long epoch = dateTime.getTime() / 1000L;

   float epoch_ = (float) (epoch - begin2020);

   return epoch_ / 86400;

}



/* use the format MM dd */

static String timestampToDate(float timestamp) {

   long epoch = (long) ((timestamp * 86400 + begin2020) * 1000);

   Date dateTime = new Date(epoch);

   SimpleDateFormat formatter = new SimpleDateFormat("MMM dd@HH:mm", Locale.ENGLISH);

   return formatter.format(dateTime);

}
```

```
}

package com.capstone.energytrade;

import android.bluetooth.BluetoothSocket;
import android.content.Intent;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;

import java.io.IOException;

public class sell_complete extends AppCompatActivity {

    private TextView text1_sellcomplete;
    private Button main_sellcomplete;
    private Button CT_sellcomplete;
    private Button stop_sellcomplete;
    private BluetoothSocket btSocket_sellcomplete =
        BluetoothApplication.getApplication().getCurrentBluetoothConnection();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_sell_complete);

        text1_sellcomplete = findViewById(R.id.text1_sellcomplete);
```

```
stop_sellcomplete = findViewById(R.id.stop_sellcomplete);

main_sellcomplete = findViewById(R.id.main_sellcomplete);

CT_sellcomplete = findViewById(R.id.CT_sellcomplete);


main_sellcomplete.setOnClickListener(new View.OnClickListener() {

    @Override

    public void onClick(View v) {

        Intent rtm_sellcomplete = new Intent(sell_complete.this, setup_interface.class);

        startActivity(rtm_sellcomplete);

    }

});


CT_sellcomplete.setOnClickListener(new View.OnClickListener() {

    @Override

    public void onClick(View v) {

        if (btSocket_sellcomplete != null) {

            try {

                btSocket_sellcomplete.getOutputStream().write('5');

                btSocket_sellcomplete.getOutputStream().write('\n');

            } catch (IOException e) {

                msg("Error");

            }

        }

        system_status.sellStatus = true;

        finish();

    }

});


stop_sellcomplete.setOnClickListener(new View.OnClickListener() {

    @Override
```

```
            public void onClick(View v) {
                if (btSocket_sellcomplete != null) {
                    try {
                        btSocket_sellcomplete.getOutputStream().write('6');
                        btSocket_sellcomplete.getOutputStream().write('\n');
                    } catch (IOException e) {
                        msg("Error");
                    }
                }
                system_status.sellStatus = true;
                finish();
            }
        });


    }


    private void msg(String s) {
        Toast.makeText(getApplicationContext(), s, Toast.LENGTH_LONG).show();
    }
}


package com.capstone.energytrade;


import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;


import java.io.File;
import java.io.PrintWriter;
import java.io.StringWriter;
import java.util.ArrayList;
```

```
import java.util.UUID;


class DB {
    static final String DBName = "HistoryDatabase";


    static void SaveBatteryLevel(File DBPath, double level) {
        if (!DBPath.getParentFile().exists()) {
            DBPath.getParentFile().mkdirs();
        }
        SQLiteDatabase db = SQLiteDatabase.openOrCreateDatabase(DBPath, null);
        db.execSQL("create table if not exists battery(id text primary key, level real not null, " +
            "time" +
            " text not null)");
        db.execSQL(String.format("insert into battery values('%s', %.2f, '%s')",
            UUID.randomUUID().toString(), level, conversion.EncodeTimeString()));
    }


    /* if successful, return null, otherwise return stacktrace string */
    static String SaveHistory(File DBPath, String type, double amount) {
        try {
            if (!DBPath.getParentFile().exists()) {
                DBPath.getParentFile().mkdirs();
            }
            SQLiteDatabase db = SQLiteDatabase.openOrCreateDatabase(DBPath, null);
            db.execSQL("create table if not exists history(id text primary key, type text not " +
                "null, " +
                "time text not null, amount real not null);");
            /* Test Row */
            db.execSQL(String.format("insert into history values('%s', '%s', '%s', %.2f)",
                UUID.randomUUID().toString(), type, conversion.EncodeTimeString(), amount));
```

```
        return "Success";
    } catch (Exception ex) { /* print stack trace */
        StringWriter sw = new StringWriter();
        ex.printStackTrace(new PrintWriter(sw));
        return sw.toString();
    }
}

/* drops history table; if successful, return true, otherwise return false*/
static boolean DeleteDB(File DBPath) {
    try {
        if (!DBPath.getParentFile().exists()) {
            DBPath.getParentFile().mkdirs();
        }
        SQLiteDatabase db = SQLiteDatabase.openOrCreateDatabase(DBPath, null);
        db.execSQL("drop table history;");
        db.execSQL("drop table battery;");
        return true;
    } catch (Exception ex) {
        return false;
    }
}

static ArrayList<HistoryContent> GetHistoryContent(File DBPath, String query) {
    ArrayList<HistoryContent> ResultList = new ArrayList<>();
    try {
        SQLiteDatabase db = SQLiteDatabase.openOrCreateDatabase(DBPath, null);
        Cursor resultSet = db.rawQuery(query, null);
        while (resultSet.moveToNext()) {
            int id = resultSet.getInt(0);
```

```
            String type = resultSet.getString(1);
            String time = resultSet.getString(2);
            double amount = resultSet.getDouble(3);
            ResultList.add(new HistoryContent(id, type, time, amount));
        }
        resultSet.close();
    } catch (Exception ex) {
        System.exit(0);
    }
    return ResultList;
}

static ArrayList<BatteryContent> GetBatteryContent(File DBPath, String query) {
    ArrayList<BatteryContent> ResultList = new ArrayList<>();
    try {
        SQLiteDatabase db = SQLiteDatabase.openOrCreateDatabase(DBPath, null);
        Cursor resultSet = db.rawQuery(query, null);
        while (resultSet.moveToNext()) {
            int id = resultSet.getInt(0);
            double level = resultSet.getDouble(1);
            String time = resultSet.getString(2);
            ResultList.add(new BatteryContent(id, level, time));
        }
        resultSet.close();
    } catch (Exception ex) {
        System.exit(0);
    }
    return ResultList;
}
```

```
static class BatteryContent {

    private final int id;

    private final double level;

    private final String time;


    public BatteryContent(int id, double level, String time) {

        this.id = id;

        this.level = level;

        this.time = time;

    }


    public int getId() {

        return id;

    }


    public double getLevel() {

        return level;

    }


    public String getTime() {

        return time;

    }

}


static class HistoryContent {

    private final int id;

    private final String type;

    private final String time;

    private final double amount;
```

```
        private HistoryContent(int id, String type, String time, double amount) {
            this.id = id;
            this.type = type;
            this.time = time;
            this.amount = amount;
        }

        public int getId() {
            return id;
        }

        public String getType() {
            return type;
        }

        public String getTime() {
            return time;
        }

        public double getAmount() {
            return amount;
        }
    }

}

package com.capstone.energytrade;

import android.bluetooth.BluetoothSocket;
import android.content.Intent;
```

```
import android.os.Bundle;

import android.support.v7.app.AppCompatActivity;

import android.view.View;

import android.widget.Button;

import android.widget.EditText;

import android.widget.ImageButton;

import android.widget.Toast;


import java.io.File;

import java.io.IOException;


public class sell_interface extends AppCompatActivity {


    private ImageButton return_sell;

    private Button sell_sell;


    private EditText price_sell;

    private EditText quantity_sell;


    private BluetoothSocket btSocket_sell =

        BluetoothApplication.getApplication().getCurrentBluetoothConnection();

    private File DBPath = this.getDatabasePath(DB.DBName);


    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_sell_interface);


        return_sell = findViewById(R.id.return_sell);

        sell_sell = findViewById(R.id.sell_sell);
```

```
price_sell = findViewById(R.id.price_sell);

quantity_sell = findViewById(R.id.quantity_sell);


return_sell.setOnClickListener(new View.OnClickListener() {

    @Override

    public void onClick(View v) {

        finish();

    }

});


sell_sell.setOnClickListener(new View.OnClickListener() {

    @Override

    public void onClick(View v) {

        if (btSocket_sell != null) {

            try {

                btSocket_sell.getOutputStream().write('0');

                btSocket_sell.getOutputStream().write('/');

                btSocket_sell.getOutputStream().write(price_sell.getText().toString().getBytes());

                btSocket_sell.getOutputStream().write('/');

                btSocket_sell.getOutputStream().write(quantity_sell.getText().toString().getBytes());

                btSocket_sell.getOutputStream().write('\n');

                /* save record to db */

                DB.SaveHistory(DBPath, "SELL",

                    Double.parseDouble(price_sell.getText().toString()));

            } catch (IOException e) {

                msg("Error");

            }

        }


        system_status.sellStatus = false;
```

```
            Intent nextpage_sell = new Intent(sell_interface.this, sell_complete.class);
            startActivity(nextpage_sell);
        }
    });

  }

  private void msg(String s) {
    Toast.makeText(getApplicationContext(), s, Toast.LENGTH_LONG).show();
  }
}

package com.capstone.energytrade;

import android.bluetooth.BluetoothSocket;
import android.content.Intent;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.view.View;
import android.widget.Button;
import android.widget.ImageButton;
import android.widget.ProgressBar;
import android.widget.TextView;
import android.widget.Toast;

import java.io.IOException;
import java.io.InputStream;
import java.util.Timer;
import java.util.TimerTask;
```

```
import static com.capstone.energytrade.DB.DBName;



public class setup_interface extends AppCompatActivity {


    private ImageButton return_setup;

    private Button btb_setup;

    private Button btu_setup;

    private Button sell_setup;

    private ImageButton setup_setup;

    private Button history;


    private TextView storage_setup;


    private InputStream stateofcharge = null;

    private ProgressBar progress_setup;


    private BluetoothSocket btSocket_setup = null;

    private InputStream inStream_setup = null;


    @Override

    protected void onCreate(Bundle savedInstanceState) {

//      try { btSocket_setup.getOutputStream().write('A'); } catch (IOException ex) { ex.printStackTrace(); System.exit(0);}

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_setup_interface);

        Intent newint = getIntent();

        btSocket_setup = BluetoothApplication.getApplication().getCurrentBluetoothConnection();

        auto_transfer.autoTransferThread(this.getDatabasePath(DBName), 100000L, true,

            BluetoothApplication.getApplication().getCurrentBluetoothConnection()).start();

//      System.out.println(btSocket_setup == null);
```

```
//      System.exit(0);

        return_setup = findViewById(R.id.return_setup);

        btb_setup = findViewById(R.id.btb_setup);

        btu_setup = findViewById(R.id.btu_setup);

        sell_setup = findViewById(R.id.sell_setup);

        setup_setup = findViewById(R.id.setup_setup);

        storage_setup = findViewById(R.id.storage_setup);

        progress_setup = findViewById(R.id.progress_setup);

        history = findViewById(R.id.history);




        requestStateofCharge();




        history.setOnClickListener(new View.OnClickListener() {

            @Override

            public void onClick(View v) {

                Intent i = new Intent(setup_interface.this, transaction_history.class);

                startActivity(i);

            }

        });


        return_setup.setOnClickListener(new View.OnClickListener() {

            @Override

            public void onClick(View v) {

                Intent returntomain_setup = new Intent(setup_interface.this, btPairing.class);

                startActivity(returntomain_setup);

            }

        });
```

```
btb_setup.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        if (system_status.btbStatus) {
            Intent btbInterface = new Intent(setup_interface.this, tradeEnergy.class);
            startActivity(btbInterface);
        } else {
            Intent btbcInterface = new Intent(setup_interface.this,
                buytobattery_complete.class);
            startActivity(btbcInterface);
        }


    }
});


btu_setup.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        if (system_status.btuStatus) {
            Intent btuInterface = new Intent(setup_interface.this, buytouse.class);
            startActivity(btuInterface);
        } else {
            Intent btucInterface = new Intent(setup_interface.this,
                buytouse_complete.class);
            startActivity(btucInterface);
        }


    }
});
```

```
    sell_setup.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            if (system_status.sellStatus) {
                Intent sellInterface = new Intent(setup_interface.this, sell_interface.class);
                startActivity(sellInterface);


            } else {
                Intent sellcomplete = new Intent(setup_interface.this, sell_complete.class);
                startActivity(sellcomplete);
            }
        }
    });


    setup_setup.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            Intent systemSetUp = new Intent(setup_interface.this, system_setup.class);
            startActivity(systemSetUp);
        }
    });
}


private void requestStateofCharge() {
    if (btSocket_setup != null) {
        try {
            btSocket_setup.getOutputStream().write('7');
            btSocket_setup.getOutputStream().write('\n');
            Timer timer = new Timer();
```

```
timer.schedule(new TimerTask() {
    @Override

    public void run() {
        runOnUiThread(new Runnable() {
            @Override
            public void run() {
                try {
                    inStream_setup = btSocket_setup.getInputStream();
                    int inStreamAvailable = inStream_setup.available();
                    if (inStreamAvailable > 0) {
                        byte[] packetBytes_setup = new byte[inStreamAvailable];
                        inStream_setup.read(packetBytes_setup);
                        System.out.println(packetBytes_setup);
                        String[] socInfo = new String(packetBytes_setup).split("/");
                        String stateofcharge = socInfo[1].trim();
                        System.out.println(socInfo);
                        storage_setup.setText(stateofcharge + "%");

                        progress_setup.setProgress(Math.round(Float.parseFloat(stateofcharge)));
                    }
                } catch (IOException E) {
                    msg("Error");
                }
            }
        });

    }

}, 1000);//Update text every second
```

```
        } catch (IOException E) {
            msg("Error");
        }
    }
  }

  private void msg(String s) {
    Toast.makeText(getApplicationContext(), s, Toast.LENGTH_LONG).show();
  }

}

package com.capstone.energytrade;

import android.bluetooth.BluetoothSocket;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ImageButton;
import android.widget.TextView;
import android.widget.Toast;

import java.io.IOException;

public class system_setup extends AppCompatActivity {

  private Button update_system;
```

```
private ImageButton return_system;

private EditText myport_system;

private EditText batterysize_system;

private EditText port1_system;

private EditText port2_system;

private EditText port3_system;

private TextView text1_system;

private TextView text2_system;

private TextView text3_system;

private TextView text4_system;

private TextView text5_system;

private TextView text6_system;

private BluetoothSocket btSocket_system =

    BluetoothApplication.getApplication().getCurrentBluetoothConnection();



@Override

protected void onCreate(Bundle savedInstanceState) {

    super.onCreate(savedInstanceState);

    setContentView(R.layout.activity_system_setup);


    update_system = findViewById(R.id.update_system);

    return_system = findViewById(R.id.return_system);

    batterysize_system = findViewById(R.id.batterysize_system);

    myport_system = findViewById(R.id.myport_system);

    port1_system = findViewById(R.id.port1_system);

    port2_system = findViewById(R.id.port2_system);

    port3_system = findViewById(R.id.port3_system);
```

```
    return_system.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            finish();
        }
    });

    update_system.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            systemUpdate();
        }
    });

}

private void systemUpdate() {
    if (btSocket_system != null) {
        try {
            btSocket_system.getOutputStream().write('3');
            btSocket_system.getOutputStream().write('/');

            btSocket_system.getOutputStream().write(batterysize_system.getText().toString().getBytes());
            btSocket_system.getOutputStream().write('/');

            btSocket_system.getOutputStream().write(myport_system.getText().toString().getBytes());
            btSocket_system.getOutputStream().write('/');

            if (port1_system.getText().toString().trim().equals("")) {
                btSocket_system.getOutputStream().write('0');
```

```
            btSocket_system.getOutputStream().write('0');


        } else {

            btSocket_system.getOutputStream().write(port1_system.getText().toString().trim().getBytes());

        }

        btSocket_system.getOutputStream().write('/');


        if (port2_system.getText().toString().trim().equals("")) {

            btSocket_system.getOutputStream().write('0');

            btSocket_system.getOutputStream().write('0');


        } else {

            btSocket_system.getOutputStream().write(port2_system.getText().toString().trim().getBytes());

        }

        btSocket_system.getOutputStream().write('/');


        if (port3_system.getText().toString().trim().equals("")) {

            btSocket_system.getOutputStream().write('0');

            btSocket_system.getOutputStream().write('0');


        } else {

            btSocket_system.getOutputStream().write(port3_system.getText().toString().trim().getBytes());

        }

        btSocket_system.getOutputStream().write('\n');


    } catch (IOException e) {

        msg("Error");

    }

  }

}
```

```
    private void msg(String s) {

        Toast.makeText(getApplicationContext(), s, Toast.LENGTH_LONG).show();

    }

}


package com.capstone.energytrade;


import android.support.v7.app.AppCompatActivity;


public class system_status extends AppCompatActivity {


    public static boolean btbStatus = true;

    public static boolean btuStatus = true;

    public static boolean sellStatus = true;


}


package com.capstone.energytrade;


import android.bluetooth.BluetoothSocket;

import android.content.Intent;

import android.os.Bundle;

import android.support.v7.app.AppCompatActivity;

import android.view.View;

import android.widget.AdapterView;

import android.widget.AdapterView.OnItemClickListener;

import android.widget.ArrayAdapter;

import android.widget.Button;

import android.widget.EditText;
```

```
import android.widget.ImageButton;

import android.widget.ListView;

import android.widget.TextView;

import android.widget.Toast;


import java.io.IOException;

import java.io.InputStream;

import java.util.ArrayList;

import java.util.HashMap;

import java.util.Map;

import java.util.Timer;

import java.util.TimerTask;


public class tradeEnergy extends AppCompatActivity {


    int readBufferPosition = 0;

    byte[] readBuffer = new byte[1024];

    private Map<Integer, Map<String, String>> TRANSACTION_INFO;

    private Integer nodeIDSelected;

    private ImageButton return_tradeenergy;

    private Button update_tradeenergy;

    private Button btb_tradeenergy;

    private TextView text1_tradeenergy;

    private TextView text2_tradeenergy;

    private TextView text3_tradeenergy;

    private TextView price_tradeenergy;

    private EditText quantity_tradeenergy;

    private ListView offerlist_tradeenergy;

    private BluetoothSocket btSocket_tradeenergy =

        BluetoothApplication.getApplication().getCurrentBluetoothConnection();
```

```
private InputStream inStream = null;

private OnItemClickListener myOfferListClickListener = new OnItemClickListener() {

    @Override

    public void onItemClick(AdapterView<?> parent, View view, int position, long id) {

        String transactionInfo = ((TextView) view).getText().toString();

        System.out.println(transactionInfo);

        String[] extractNodeID = transactionInfo.split("\n");

        int householdID = Integer.parseInt(extractNodeID[0].replaceAll("[^0-9]", ""));

        Map<String, String> offerDetails = TRANSACTION_INFO.get(householdID);

        price_tradeenergy.setText(offerDetails.get("Price"));

        quantity_tradeenergy.setText(offerDetails.get("Quantity"));

        nodeIDSelected = householdID;


    }

};


@Override

protected void onCreate(Bundle savedInstanceState) {

    super.onCreate(savedInstanceState);

    setContentView(R.layout.activity_trade_energy);


    return_tradeenergy = findViewById(R.id.return_tradeenergy);

    btb_tradeenergy = findViewById(R.id.btb_tradeenergy);

    update_tradeenergy = findViewById(R.id.update_tradeenergy);


    quantity_tradeenergy = findViewById(R.id.quantity_tradeenergy);

    price_tradeenergy = findViewById(R.id.price_tradeenergy);

    offerlist_tradeenergy = findViewById(R.id.offerlist_tradeenergy);

    return_tradeenergy.setOnClickListener(new View.OnClickListener() {

        @Override
```

```
        public void onClick(View v) {
            finish();
        }
    });

    update_tradeenergy.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            receiveUpdates();
        }
    });

    btb_tradeenergy.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            confirmTransaction();
            system_status.btbStatus = false;
            Intent btbc = new Intent(tradeEnergy.this, buytobattery_complete.class);
            startActivity(btbc);

        }
    });

}

private void receiveUpdates() {
    if (btSocket_tradeenergy != null) {
        try {
            btSocket_tradeenergy.getOutputStream().write('1');
```

```
btSocket_tradeenergy.getOutputStream().write('\n');


Timer timer = new Timer();
timer.schedule(new TimerTask() {
    @Override
    public void run() {
        runOnUiThread(new Runnable() {
            @Override
            public void run() {
                try {
                    inStream = btSocket_tradeenergy.getInputStream();
                    int bytesAvailable = inStream.available();
                    if (bytesAvailable > 0) {
                        byte[] packetBytes = new byte[bytesAvailable];
                        System.out.println(bytesAvailable);
                        inStream.read(packetBytes);


                        for (int i = 0; i < bytesAvailable; i++) {
                            byte b = packetBytes[i];
                            if (b != 0) {
                                readBuffer[readBufferPosition++] = b;
                                System.out.println((char) b);
                            }
                            System.out.println(new String(readBuffer));
                        }
                        byte[] encodedBytes = new byte[readBufferPosition];
                        System.arraycopy(readBuffer, 0, encodedBytes, 0,
                            encodedBytes.length);


                        String[] offerOption = new String(encodedBytes).split("\n");
```

```
ArrayList<String> offer_list = new ArrayList<String>();

TRANSACTION_INFO = new HashMap<Integer, Map<String,

    String>>();

for (int i = 0; i < offerOption.length; i++) {

    String[] offerInformation = offerOption[i].split("/");



    if (offerInformation.length >= 6) {

        String signal = offerInformation[0];

        System.out.println(signal);

        String nodeID = offerInformation[1];

        System.out.println(nodeID);

        String price = offerInformation[2];

        System.out.println(price);

        String quantity = offerInformation[3];

        System.out.println(quantity);

        String count = offerInformation[4];

        System.out.println(count);

        String path = offerInformation[5];

        System.out.println(path);



        Map<String, String> offerDetails =

            new HashMap<String, String>();

        offerDetails.put("Price", price);

        offerDetails.put("Quantity", quantity);

        offerDetails.put("Count", count);

        offerDetails.put("Path", path);



        int householdID = Integer.parseInt(nodeID);

        TRANSACTION_INFO.put(householdID, offerDetails);

        offer_list.add("Node ID:" + nodeID + '\n' +
```

```
                    "Price:" + price + "$/Ah" +
                    " Quantity:" + quantity + "Ah");


                }
            }
            final ArrayAdapter<String> arrayAdapter =
                new ArrayAdapter<String>(tradeEnergy.this,
                    android.R.layout.simple_list_item_1, offer_list);
            offerlist_tradeenergy.setAdapter(arrayAdapter);
            offerlist_tradeenergy.setOnItemClickListener(myOfferListClickListener);


            }
        } catch (IOException ex) {


        }


        }
    });
    }
  }, 2000);


  } catch (IOException e) {
    e.printStackTrace();
  }
 }
}


private void confirmTransaction() {
 if (btSocket_tradeenergy != null) {
  try {
```

```
btSocket_tradeenergy.getOutputStream().write('2');
btSocket_tradeenergy.getOutputStream().write('/');



if (nodeIDSelected < 10) {
    btSocket_tradeenergy.getOutputStream().write('0');
    btSocket_tradeenergy.getOutputStream().write(nodeIDSelected.toString().getBytes());
} else {
    btSocket_tradeenergy.getOutputStream().write(nodeIDSelected.toString().getBytes());
}


btSocket_tradeenergy.getOutputStream().write('/');


btSocket_tradeenergy.getOutputStream().write(TRANSACTION_INFO.get(nodeIDSelected).get(
    "Price").getBytes());
btSocket_tradeenergy.getOutputStream().write('/');


btSocket_tradeenergy.getOutputStream().write(quantity_tradeenergy.getText().toString().getBytes());
btSocket_tradeenergy.getOutputStream().write('/');


btSocket_tradeenergy.getOutputStream().write(TRANSACTION_INFO.get(nodeIDSelected).get(
    "Count").getBytes());
btSocket_tradeenergy.getOutputStream().write('/');


btSocket_tradeenergy.getOutputStream().write(TRANSACTION_INFO.get(nodeIDSelected).get(
    "Path").getBytes());


btSocket_tradeenergy.getOutputStream().write('\n');


/* write to db */
```

```
//      DB.SaveHistory(this.getDatabasePath(DB.DBName), "BUY", Double.parseDouble
//      (quantity_tradeenergy.getText().toString()));


        } catch (IOException e) {
            msg("Error");
        }
    }
  }


  private void msg(String s) {
    Toast.makeText(getApplicationContext(), s, Toast.LENGTH_LONG).show();
  }


}
package com.capstone.energytrade;


import android.content.Intent;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.ListView;


import java.util.ArrayList;


public class transaction_history extends AppCompatActivity {


  private View.OnClickListener forwardGraphViewHistory = new View.OnClickListener() {
    @Override
```

```
    public void onClick(View view) {
        Intent i = new Intent(transaction_history.this, transaction_history_graph.class);
        startActivity(i);
    }
};
private View.OnClickListener forwardGraphViewBattery = new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        Intent i = new Intent(transaction_history.this, battery_level_graph.class);
        startActivity(i);
    }
};

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_transaction_history);
    render();
}

private void render() {
    final ListView transactionList = findViewById(R.id.transactionHistory);
    final Button graphViewHistory = findViewById(R.id.graphViewHistory);
    final Button graphViewBattery = findViewById(R.id.graphViewBattery);
    ArrayList<String> TList = new ArrayList<>();
    final ArrayAdapter myAdapter = new ArrayAdapter(this, android.R.layout.simple_list_item_1,
        TList);

    ArrayList<DB.HistoryContent> transactions =
        DB.GetHistoryContent(this.getDatabasePath(DB.DBName), "select * from history");
```

```
        for (DB.HistoryContent t : transactions) {
            myAdapter.add(t.getType() + ", " + t.getAmount() + " Watts" + ", " + t.getTime());
        }
        graphViewHistory.setOnClickListener(forwardGraphViewHistory);
        graphViewBattery.setOnClickListener(forwardGraphViewBattery);
        transactionList.setAdapter(myAdapter);
    }
}
package com.capstone.energytrade;


import android.content.Intent;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.ListView;


import java.util.ArrayList;


public class transaction_history extends AppCompatActivity {


    private View.OnClickListener forwardGraphViewHistory = new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            Intent i = new Intent(transaction_history.this, transaction_history_graph.class);
            startActivity(i);
        }
    };
    private View.OnClickListener forwardGraphViewBattery = new View.OnClickListener() {
```

```
    @Override
    public void onClick(View view) {
        Intent i = new Intent(transaction_history.this, battery_level_graph.class);
        startActivity(i);
    }
};

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_transaction_history);
    render();
}

private void render() {
    final ListView transactionList = findViewById(R.id.transactionHistory);
    final Button graphViewHistory = findViewById(R.id.graphViewHistory);
    final Button graphViewBattery = findViewById(R.id.graphViewBattery);
    ArrayList<String> TList = new ArrayList<>();
    final ArrayAdapter myAdapter = new ArrayAdapter(this, android.R.layout.simple_list_item_1,
        TList);

    ArrayList<DB.HistoryContent> transactions =
        DB.GetHistoryContent(this.getDatabasePath(DB.DBName), "select * from history");
    for (DB.HistoryContent t : transactions) {
        myAdapter.add(t.getType() + ", " + t.getAmount() + " Watts" + ", " + t.getTime());
    }
    graphViewHistory.setOnClickListener(forwardGraphViewHistory);
    graphViewBattery.setOnClickListener(forwardGraphViewBattery);
    transactionList.setAdapter(myAdapter);
```

```
    }
}

package com.capstone.energytrade;

import android.content.Intent;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.ListView;

import java.util.ArrayList;

public class transaction_history extends AppCompatActivity {

    private View.OnClickListener forwardGraphViewHistory = new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            Intent i = new Intent(transaction_history.this, transaction_history_graph.class);
            startActivity(i);
        }
    };
    private View.OnClickListener forwardGraphViewBattery = new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            Intent i = new Intent(transaction_history.this, battery_level_graph.class);
            startActivity(i);
        }
```

```
    };

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_transaction_history);
        render();
    }

    private void render() {
        final ListView transactionList = findViewById(R.id.transactionHistory);
        final Button graphViewHistory = findViewById(R.id.graphViewHistory);
        final Button graphViewBattery = findViewById(R.id.graphViewBattery);
        ArrayList<String> TList = new ArrayList<>();
        final ArrayAdapter myAdapter = new ArrayAdapter(this, android.R.layout.simple_list_item_1,
            TList);

        ArrayList<DB.HistoryContent> transactions =
            DB.GetHistoryContent(this.getDatabasePath(DB.DBName), "select * from history");
        for (DB.HistoryContent t : transactions) {
            myAdapter.add(t.getType() + ", " + t.getAmount() + " Watts" + ", " + t.getTime());
        }
        graphViewHistory.setOnClickListener(forwardGraphViewHistory);
        graphViewBattery.setOnClickListener(forwardGraphViewBattery);
        transactionList.setAdapter(myAdapter);
    }
}

package com.capstone.energytrade;

import android.content.Intent;
```

```
import android.os.Bundle;

import android.support.v7.app.AppCompatActivity;

import android.view.View;

import android.widget.ArrayAdapter;

import android.widget.Button;

import android.widget.ListView;


import java.util.ArrayList;


public class transaction_history extends AppCompatActivity {


    private View.OnClickListener forwardGraphViewHistory = new View.OnClickListener() {

        @Override

        public void onClick(View view) {

            Intent i = new Intent(transaction_history.this, transaction_history_graph.class);

            startActivity(i);

        }

    };

    private View.OnClickListener forwardGraphViewBattery = new View.OnClickListener() {

        @Override

        public void onClick(View view) {

            Intent i = new Intent(transaction_history.this, battery_level_graph.class);

            startActivity(i);

        }

    };


    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_transaction_history);
```

```
        render();
    }

    private void render() {
        final ListView transactionList = findViewById(R.id.transactionHistory);
        final Button graphViewHistory = findViewById(R.id.graphViewHistory);
        final Button graphViewBattery = findViewById(R.id.graphViewBattery);
        ArrayList<String> TList = new ArrayList<>();
        final ArrayAdapter myAdapter = new ArrayAdapter(this, android.R.layout.simple_list_item_1,
            TList);

        ArrayList<DB.HistoryContent> transactions =
            DB.GetHistoryContent(this.getDatabasePath(DB.DBName), "select * from history");
        for (DB.HistoryContent t : transactions) {
            myAdapter.add(t.getType() + ", " + t.getAmount() + " Watts" + ", " + t.getTime());
        }
        graphViewHistory.setOnClickListener(forwardGraphViewHistory);
        graphViewBattery.setOnClickListener(forwardGraphViewBattery);
        transactionList.setAdapter(myAdapter);
    }
}
package com.capstone.energytrade;

import android.content.Intent;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.ListView;
```

```
import java.util.ArrayList;


public class transaction_history extends AppCompatActivity {


    private View.OnClickListener forwardGraphViewHistory = new View.OnClickListener() {

        @Override

        public void onClick(View view) {

            Intent i = new Intent(transaction_history.this, transaction_history_graph.class);

            startActivity(i);

        }

    };

    private View.OnClickListener forwardGraphViewBattery = new View.OnClickListener() {

        @Override

        public void onClick(View view) {

            Intent i = new Intent(transaction_history.this, battery_level_graph.class);

            startActivity(i);

        }

    };


    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_transaction_history);

        render();

    }


    private void render() {

        final ListView transactionList = findViewById(R.id.transactionHistory);

        final Button graphViewHistory = findViewById(R.id.graphViewHistory);
```

```
        final Button graphViewBattery = findViewById(R.id.graphViewBattery);

        ArrayList<String> TList = new ArrayList<>();

        final ArrayAdapter myAdapter = new ArrayAdapter(this, android.R.layout.simple_list_item_1,

            TList);


        ArrayList<DB.HistoryContent> transactions =

            DB.GetHistoryContent(this.getDatabasePath(DB.DBName), "select * from history");

        for (DB.HistoryContent t : transactions) {

            myAdapter.add(t.getType() + ", " + t.getAmount() + " Watts" + ", " + t.getTime());

        }

        graphViewHistory.setOnClickListener(forwardGraphViewHistory);

        graphViewBattery.setOnClickListener(forwardGraphViewBattery);

        transactionList.setAdapter(myAdapter);

    }

}

package com.capstone.energytrade;


import android.content.Intent;

import android.os.Bundle;

import android.support.v7.app.AppCompatActivity;

import android.view.View;

import android.widget.ArrayAdapter;

import android.widget.Button;

import android.widget.ListView;


import java.util.ArrayList;


public class transaction_history extends AppCompatActivity {


    private View.OnClickListener forwardGraphViewHistory = new View.OnClickListener() {
```

```
    @Override
    public void onClick(View view) {
        Intent i = new Intent(transaction_history.this, transaction_history_graph.class);
        startActivity(i);
    }
};
private View.OnClickListener forwardGraphViewBattery = new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        Intent i = new Intent(transaction_history.this, battery_level_graph.class);
        startActivity(i);
    }
};


@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_transaction_history);
    render();
}


private void render() {
    final ListView transactionList = findViewById(R.id.transactionHistory);
    final Button graphViewHistory = findViewById(R.id.graphViewHistory);
    final Button graphViewBattery = findViewById(R.id.graphViewBattery);
    ArrayList<String> TList = new ArrayList<>();
    final ArrayAdapter myAdapter = new ArrayAdapter(this, android.R.layout.simple_list_item_1,
        TList);


    ArrayList<DB.HistoryContent> transactions =
```

```
        DB.GetHistoryContent(this.getDatabasePath(DB.DBName), "select * from history");

      for (DB.HistoryContent t : transactions) {

        myAdapter.add(t.getType() + ", " + t.getAmount() + " Watts" + ", " + t.getTime());

      }

      graphViewHistory.setOnClickListener(forwardGraphViewHistory);

      graphViewBattery.setOnClickListener(forwardGraphViewBattery);

      transactionList.setAdapter(myAdapter);

    }

}

package com.capstone.energytrade;


import android.graphics.Color;

import android.os.Bundle;

import android.support.v7.app.AppCompatActivity;


import com.github.mikephil.charting.charts.LineChart;

import com.github.mikephil.charting.components.XAxis;

import com.github.mikephil.charting.components.YAxis;

import com.github.mikephil.charting.data.Entry;

import com.github.mikephil.charting.data.LineData;

import com.github.mikephil.charting.data.LineDataSet;

import com.github.mikephil.charting.formatter.ValueFormatter;

import com.github.mikephil.charting.interfaces.datasets.ILineDataSet;


import java.util.ArrayList;

import java.util.List;

import java.util.stream.Collectors;


public class transaction_history_graph extends AppCompatActivity {
```

```
List<DataPoint> buy_data;

List<DataPoint> sell_data;


@Override

protected void onCreate(Bundle savedInstanceState) {

    super.onCreate(savedInstanceState);

    setContentView(R.layout.activity_transaction_history_graph);

    loadData();

    render();

}


private void loadData() {

    ArrayList<DB.HistoryContent> buy_contents =

        DB.GetHistoryContent(this.getDatabasePath(DB.DBName), "select * from history where " +

            "type = " +

            "'BUY' or type = 'AUTO_BUY'");

    ArrayList<DB.HistoryContent> sell_contents =

        DB.GetHistoryContent(this.getDatabasePath(DB.DBName), "select * from history where " +

            "type = " +

            "'SELL'");

    buy_data =

        buy_contents.stream().map(b_d -> new DataPoint(b_d.getTime(), b_d.getAmount())).collect(Collectors.toList());

    sell_data =

        sell_contents.stream().map(s_d -> new DataPoint(s_d.getTime(), s_d.getAmount())).collect(Collectors.toList());

}


private void render() {

    LineChart chart = findViewById(R.id.chart);


    /* format axis */
```

```
XAxis xAxis = chart.getXAxis();

YAxis leftAxis = chart.getAxisLeft();

YAxis rightAxis = chart.getAxisRight();


/* customize xaxis */

xAxis.setPosition(XAxis.XAxisPosition.BOTTOM);

xAxis.setDrawGridLines(false);

xAxis.setDrawAxisLine(false);

xAxis.setValueFormatter(new ValueFormatter() {

   @Override

   public String getFormattedValue(float value) {

      return conversion.timestampToDate(value);

   }

});


/* customize yaxis */

leftAxis.setDrawGridLines(false);

leftAxis.setDrawAxisLine(false);

rightAxis.setEnabled(false);


List<Entry> buy_entries = new ArrayList<>();

List<Entry> sell_entries = new ArrayList<>();

for (DataPoint dp : buy_data) {

   float timestamp = conversion.dateToTimestamp(dp.date);

   if (timestamp != -1) {

      buy_entries.add(new Entry(conversion.dateToTimestamp(dp.date), (float) dp.amount));

   }

}

for (DataPoint dp : sell_data) {

   float timestamp = conversion.dateToTimestamp(dp.date);
```

```
            if (timestamp != -1) {
                sell_entries.add(new Entry(conversion.dateToTimestamp(dp.date), (float) dp.amount));
            }
        }
        LineDataSet buy_dataSet = new LineDataSet(buy_entries, "Transactions-Buy");
        buy_dataSet.setAxisDependency(YAxis.AxisDependency.LEFT);
        LineDataSet sell_dataSet = new LineDataSet(sell_entries, "Transactions-Sell");
        sell_dataSet.setAxisDependency(YAxis.AxisDependency.LEFT);
        buy_dataSet.setColor(Color.RED);
        sell_dataSet.setColor(Color.BLUE);
        buy_dataSet.setValueTextColor(Color.RED);
        sell_dataSet.setValueTextColor(Color.BLUE);


        List<ILineDataSet> dataSets = new ArrayList<>();
        dataSets.add(buy_dataSet);
        dataSets.add(sell_dataSet);
        LineData lineData = new LineData(dataSets);
        chart.setData(lineData);
//      chart.setBackgroundColor(Color.rgb(150, 150, 150));
        chart.invalidate();
    }


    class DataPoint {
        String date;
        double amount;


        public DataPoint(String date, double amount) {
            this.date = date;
            this.amount = amount;
        }
```

```
  }
}
```

Appendix E: Arduino code

```
#include <RF24.h>

#include <SPI.h>

#define REQUEST 0

#define RESPONSE 1

#define CONFIRMATION 2

#define CANCELLATION 3

#define IDLE_STATE 0

#define CONSUMING 1

#define GENERATING 2

#define FORWARDING 3

#define BUYING_FOR_USE 4

#define UPDATE_OFFERS '0'

#define REQUEST_ENERGY '1'

#define CONFIRM_ENERGY '2'

#define SETUP_SYSTEM '3'

#define BUY_FOR_USE '4'

#define CANCEL '5'

#define WITHDRAW_OFFER '6'

#define UPDATE_PHONE_ON_SOC '7'

/* ################### GLOBAL VARIABLES ################### */
```

```
RF24 radio(10, 9); // CE, CSN

char node_id[3] = "02";

char neighbour_ids[3][3] = {"01", "03", "04"};

byte addresses[3][2][6] = {{"00201", "00102"},

  {"00003", "00004"},

  {"00005", "00006"}

}; //write and read for three connections


const int SWITCHBOX[3] = {11, 12, 13}; // pins for controlling status of switches with three neighbours

const int SWITCHBOX_SOLAR = 8;//太阳能电池开关针脚


bool switchbox_status[3] = {0, 0, 0};

float price = 3.1;//[] = {1,0,0};

float quantity = 0;//[] = {2.4,0,0};

byte state = 0; // 0-idle, 1-consuming, 2-generating, 3-forwarding

const char terminator = '\n';

float max_current = 2.1;

float ref_voltage = 48.0;

float ref_current = 0;

float battery_capacity = 100;

float battery_charge = 0;

unsigned long t;

unsigned long currentTimer;

float purchased_energy = 0;

unsigned long transaction_timer = 0;

float converter_voltage_reading = 13.1;

float converter_current_reading = 0;

float current_sensor_reading = 0;
```

```
bool debugging = 1;


/* ##################### DATA STRUCUTRES ##################### */

struct RF_ping {

  char destination[3] ;//= "02" ; // intended recipient of this ping

  byte type = REQUEST; // 0: request, 1: response, 2: confirmation, 3: cancellation

  byte count = 1; // the address of the node receiving this ping in the path (zero indexed!)

  char path[5][3] = {"00", "00", "00", "00", "00"}; // path of node IDs through which this ping has passed. 0th element is consumer, final
used element (at i = count) is supplier

  float price = 0;//[3] {0,0,0};  // price of energy (up to three increments)

  float quantity = 0;//[3] {0,0,0}; // quantity of energy (up to three increments)*/

};


RF_ping info; // for use during testing


struct transaction_list {

  RF_ping list[10]; //the data structures for ongoing transactions will be stored here

  int count = 0;  //this specifies the number of used elements in list

};


transaction_list transactions;


/* ##################### FUNCTIONS #####################  */

/**

 * 接收无线电模块发送的数据，并做出不同的处理

 */

void receive_radio() {

  /*

    This function gets called if the radio is available. It triages the different types

    of radio communication to their respective functions where they are parsed
```

```
  */
  RF_ping data;

  radio.read(&data, sizeof(data));//读取标准数据结构体的数据

  if (debugging) {//调试模式
    Serial.print("receiving radio of type: ");
    Serial.print(data.type);
    Serial.print("\n");
  }

  if (String(data.destination).equals(String(node_id))) {//判断发送数据的地址，如果不是本节点数据则跳过
    if (data.type == REQUEST) {
      receive_request(&data);
    }
    else if (data.type == RESPONSE) {
      receive_response(&data);
    }
    else if (data.type == CONFIRMATION) {
      receive_confirmation(&data);
    }
    else if (data.type == CANCELLATION) {
      receive_cancellation(&data);
    }
  }
}

void receive_request(RF_ping *data) {
  if (quantity != 0) {
    //do this if this node has energy available to sell
```

```
  radio.stopListening();

  RF_ping response = *data;

  response.type = RESPONSE;

  strcpy(response.destination, (*data).path[(*data).count - 1]); // send to neighbour immediately before this on in the path

  strcpy(response.path[response.count], node_id); // appending this node's ID onto the path

  response.quantity = quantity;

  response.price = price;

  radio.openWritingPipe(addresses[0][0]);//[find_index((*data).path[(*data).count - 1],neighbour_ids)][0]);

  digitalWrite(LED_BUILTIN, HIGH);

  delay(100);

  bool success = 0;

  success = persistant_radio_write(&response);

  //success = persistant_radio_write(&response);

  //Serial.print("send status:");

  //Serial.print(success);

  //Serial.print("\n");

  digitalWrite(LED_BUILTIN, LOW);



  if (debugging) {

    Serial.print("sending response via node ");

    Serial.print(response.destination);

    Serial.print("\n");

  }

  print_ping(&response);

  radio.startListening();

  digitalWrite(LED_BUILTIN, LOW);

}



if ((*data).count < 5) {

  //do this if this node has no energy available to sell
```

```
//this block provides instructions as to how to forward request to neighbours


if (state == IDLE_STATE) {
  // propogate to all neighbours other than the one who sent the request
  radio.stopListening();
  digitalWrite(LED_BUILTIN, HIGH);
  for (int i; i < 3; i++) {
    if (!String(neighbour_ids[i]).equals((*data).path[(*data).count - 1])) {
      // note: the (count-1)th node in the path sent the request
      //forward to all neighbours except the one that sent it
      RF_ping new_ping = *data;
      strcpy(new_ping.destination, neighbour_ids[i]); // forwarding to neighbour i
      strcpy(new_ping.path[new_ping.count], node_id); // appending this node's ID onto the path
      new_ping.count ++; // increment count
      radio.openWritingPipe(addresses[i][0]);
      radio.write(&new_ping, sizeof(new_ping));
      if (debugging) {
        Serial.print("forwarded request to node ");
        Serial.print(new_ping.destination);
        Serial.print("\n");
      }
      print_ping(&new_ping);
    }
  }
  radio.startListening();
  digitalWrite(LED_BUILTIN, LOW);
}else if ((state == CONSUMING) or (state == FORWARDING)) {
  // forward to current switchbox connections, excluding neighbour that send it
  radio.stopListening();
  digitalWrite(LED_BUILTIN, HIGH);
```

```
    for (int i = 0; i < 3; i++) {

      if ((!String(neighbour_ids[i]).equals((*data).path[(*data).count - 1])) and (switchbox_status[i] == 1)) { // continue if neighbour id[i] is
not the node that sent the ping

        RF_ping new_ping = *data;

        strcpy(new_ping.destination, neighbour_ids[i]); // forwarding to neighbour i

        strcpy(new_ping.path[new_ping.count], node_id); // appending this node's ID onto the path

        new_ping.count ++; // increment count

        radio.openWritingPipe(addresses[i][0]);

        radio.write(&new_ping, sizeof(new_ping));

        if (debugging) {

          Serial.print("forwarded request to node ");

          Serial.print(new_ping.destination);

          Serial.print("\n");

        }

        print_ping(&new_ping);

      }

    }

    radio.startListening();

    digitalWrite(LED_BUILTIN, LOW);

  }

 }

}



void receive_response(RF_ping *data) {

 /*

   This function recieves a response to a request, and either forwards it or sends it to the phone

 */

 if (debugging) {

  Serial.print("receiving response \n");

 }
```

```
if (String((*data).path[0]).equals(String(node_id))) { // if the response is for a request for energy from this node
  //send the response to phone!
  if (debugging) {
    Serial.print("sending response to phone \n");
    print_ping(data);
  }
  Serial1.print("0/");
  Serial1.print((*data).path[(*data).count]); // sending node ID
  Serial1.print("/");
  float price_adj = (*data).price * (1 + (*data).count * 0.0029 * max_current + 0.1);
  Serial1.print(price_adj);
  Serial1.print("/");
  float quantity_adj = (*data).quantity * (1 - (*data).count * 0.0029 * max_current - 0.1);
  Serial1.print(quantity_adj);
  Serial1.print("/");
  Serial1.print((*data).count);
  Serial1.print("/");
  for (int i = 0; i < (*data).count + 1; i++) {
    Serial1.print((*data).path[i]);
    //Serial1.write("\0");
  }
  Serial1.print(terminator);


  // for testing purposes: automatically send a confirmation
  // for use if not connected to phone for user input


  /*radio.stopListening();
    digitalWrite(LED_BUILTIN,HIGH);
    delay(1000);
    print_ping(data);
```

```
    RF_ping confirmation = *data;

    confirmation.type = CONFIRMATION;

    strcpy(confirmation.destination, (*data).path[1]); // forwarding to first path element

    radio.openWritingPipe(addresses[find_index(((*data).path[1]),neighbour_ids)][0]);

    persistant_radio_write(&confirmation);

    //radio.write(&confimation,sizeof(confimation));

    if(debugging){Serial.print("confirming transaction to ");

    Serial.print(confirmation.destination);

    Serial.print("\n");}

    print_ping(&confirmation);

    radio.startListening();

    digitalWrite(LED_BUILTIN,LOW);

    state = CONSUMING;

    ref_current = max_current;

    switchbox_status[0] = 1;

    transactions.list[transactions.count] = *data;

    transactions.count ++;*/


  }else { // if this is some intermediary node in the path, for the response along

    radio.stopListening();

    digitalWrite(LED_BUILTIN, HIGH);

    int i = find_index(node_id, (*data).path); // i is now the index of this node in the path

    strcpy((*data).destination, (*data).path[i - 1]); // set destination to next node

    radio.openWritingPipe(addresses[find_index(((*data).path[i - 1]), neighbour_ids)][0]); //open writing pipe to next node

    radio.write(data, sizeof(*data)); // send the ping

    radio.startListening();

    digitalWrite(LED_BUILTIN, LOW);

    if (debugging) {

      Serial.print("forwarded response to node ");

      Serial.print((*data).destination);
```

```
      Serial.print("\n");
    }
    print_ping(data);
  }

}

void receive_confirmation(RF_ping *data) {
  /*
    this function receives a confirmation for energy, or forwards it to the required node
  */
  if (debugging) {
    Serial.print("receiving confirmation \n");
  }
  switchbox_status[0] = 0; // zero the switchbox
  switchbox_status[1] = 0;
  switchbox_status[2] = 0;
  for (int i = 0; i <= (*data).count; i++) { // iterate through the transanction path, up to the path length
    if (String((*data).path[i]).equals(String(node_id))) { // when you find this node in the path
      switchbox_status[find_index((*data).path[i - 1], neighbour_ids)] = 1; // connect the switchbox to the previous element of the path
      if (i < (*data).count) {
        switchbox_status[find_index(((*data).path[i + 1]), neighbour_ids)] = 1; // if this isn't the last node in the path, connect to next node in path
        radio.stopListening();
        digitalWrite(LED_BUILTIN, HIGH);
        strcpy((*data).destination, (*data).path[i + 1]); // set destination to next node
        radio.openWritingPipe(addresses[find_index(((*data).path[i + 1]), neighbour_ids)][0]); //open writing pipe to next node
        radio.write(data, sizeof(*data)); // send the ping
        radio.startListening();
        digitalWrite(LED_BUILTIN, LOW);
```

```
      state = FORWARDING;
      transactions.list[transactions.count] = *data;
      transactions.count ++;
    }
  }
}
if (debugging) {
  Serial.print("setting switchbox to: ");
}
for (int i = 0; i < 3; i++) {
  if (debugging) {
    Serial.print(switchbox_status[i]);
  }
}
if (debugging) {
  Serial.print("\n");
}
update_switchbox();
if (String((*data).path[(*data).count]).equals(String(node_id))) {
  // CODE TO SEND ENERGY!
  //
  transactions.list[transactions.count] = *data;
  transactions.count ++;
  if (debugging) {
    Serial.print("sending energy \n");
  }
  state = GENERATING;
}
}
```

```
void receive_cancellation(RF_ping *data) {
  state = IDLE_STATE;
  for (int i = 0; i < transactions.count; i++) {
    bool same = 0;
    byte unmatching_nodes = 0;
    for (int j = 0; j < (*data).count; j++) {
      if (~(String((*data).path[j]).equals(String(transactions.list[i].path[j])))) {
        unmatching_nodes ++;
      }
    }
    if ((unmatching_nodes == 0) && (transactions.list[i].count == (*data).count)) {
      switchbox_status[find_index(transactions.list[i].path[transactions.list[i].count - 1], neighbour_ids)] = 0;
      update_switchbox();
      for (int j = i; j < transactions.count; j++) {
        transactions.list[j] = transactions.list[j + 1];
      }
      transactions.count --;
      if (transactions.count == 0) {
        state = IDLE_STATE;
      }
      return;
    }
  }
}


/**
 * 从蓝牙收取信息
 */
void receive_BT() {
  /*
```

```
  Similar to the recieve radio function, this triages the various types of BT inputs

*/

char type;

type = Serial1.read();

if (debugging) {

  Serial.print("receiving BT \n");

  Serial.print(type);

  Serial.print("\n");

}

if (type == UPDATE_OFFERS) {

  update_offers();

}

else if (type == REQUEST_ENERGY) {

  request_energy();

}

else if (type == CONFIRM_ENERGY) {

  confirm_energy();

}

else if (type == SETUP_SYSTEM) {

  receive_setup_data();

}

else if (type == BUY_FOR_USE) {

  delay(5);

  Serial1.read();

  buy_for_use();

}

else if (type == CANCEL) {

  delay(2);

  Serial1.read();

  cancel_purchase();
```

```
  }
  else if (type == WITHDRAW_OFFER) {
    delay(2);
    Serial1.read();
    price = 0;
    quantity = 0;
  }
  else if (type == UPDATE_PHONE_ON_SOC) {
    delay(2);
    Serial1.read();
    send_state_of_charge();
  }
}


void cancel_purchase() {
  if (debugging) {
    Serial.print("Cancelling purchase \n");
  }
  ref_current = 0;
  state = IDLE_STATE;
  for (int i = 0; i < transactions.count; i++) {
    if (String(transactions.list[i].path[0]).equals(node_id)) {


      transactions.list[i].type = CANCELLATION;
      strcpy(transactions.list[i].destination, transactions.list[i].path[1]);
      radio.stopListening();
      radio.openWritingPipe(addresses[find_index(transactions.list[i].path[1], neighbour_ids)][0]);
      bool success = 0;
      success = persistant_radio_write(&(transactions.list[i]));
      radio.startListening();
```

```
      switchbox_status[find_index(transactions.list[i].path[1], neighbour_ids)] = 0;

      update_switchbox();

      if (debugging) {

        Serial.print("Cancellation sent: \n");

        print_ping(&(transactions.list[i]));

      }




      for (int j = i; j < transactions.count; j++) {

        transactions.list[j] = transactions.list[j + 1];

      }



      transactions.count --;

      return;

    }

  }

}


void update_offers() {

  delay(50);

  char temp;

  temp = Serial1.read(); //get rid of the slash

  price = Serial1.parseFloat();

  temp = Serial1.read(); //get rid of the slash

  quantity = Serial1.parseFloat();

  temp = Serial1.read(); //get rid of the terminator

}


void request_energy() {

  RF_ping info;
```

```
  radio.stopListening();

  strcpy(info.destination, neighbour_ids[0]);

  info.type = 0;

  info.count = 1;

  strcpy(info.path[0], node_id);

  if (debugging) {

    Serial.print(info.type);

    Serial.print("\n");

    Serial.print("writing request \n");

  }

  persistant_radio_write(&info);

  //radio.write(&info,sizeof(info));

  print_ping(&info);

  radio.startListening();

  Serial1.read();

}


void buy_for_use() {

  //send out a request

  RF_ping info;

  radio.stopListening();

  strcpy(info.destination, neighbour_ids[0]);

  info.type = 0;

  info.count = 1;

  strcpy(info.path[0], node_id);

  if (debugging) {

    Serial.print(info.type);

    Serial.print("\n");

    Serial.print("writing request \n");

  }
```

```
persistant_radio_write(&info);

//radio.write(&info,sizeof(info));

print_ping(&info);

radio.startListening();


RF_ping responses_adjusted_price[10]; //store responses in here

RF_ping responses[10];

int responses_received = 0;


long waiting_timer = 0;

waiting_timer = millis();

while ((millis() - waiting_timer) < 500) { // wait for responses for .5 seconds, store responses in lists responses

  if (radio.available()) {

    RF_ping data;

    radio.read(&data, sizeof(data));

    if (debugging) {

      Serial.print("receiving radio of type: ");

      Serial.print(data.type);

      Serial.print("\n");

      print_ping(&data);

    }

    if ((String(data.destination).equals(String(node_id))) && (data.type == RESPONSE)) {

      //Serial.println("added");

      responses[responses_received] = data;

      data.price = data.price * (1 + data.count * 0.0029 * max_current);

      responses_adjusted_price[responses_received] = data;

      responses_received++;

    }

  }

}
```

```
//now, of the responses received, find the cheapest one

int cheapest_response_index = 0;

float lowest_price = responses_adjusted_price[0].price;

//Serial.println(responses[0].price);

for (int i = 1; i < responses_received; i++) {

  if (responses_adjusted_price[i].price < lowest_price) {

    cheapest_response_index = i;

    lowest_price = responses_adjusted_price[i].price;

    //Serial.println(responses[i].price);

    //Serial.println(cheapest_response_index);

  }

  else {}

}

//send a confirmation to the offer with the cheapest price

//Serial.println("establishing confirmation");

if (responses_received > 0) {

  RF_ping confirmation = responses[cheapest_response_index];

  confirmation.type = CONFIRMATION;

  strcpy(confirmation.destination, confirmation.path[confirmation.count]);

  //Serial.println("sending");

  radio.stopListening();

  persistant_radio_write(&confirmation);

  radio.startListening();

  switchbox_status[find_index(confirmation.path[1], neighbour_ids)] = 1;

  update_switchbox();

  transactions.list[transactions.count] = confirmation;

  transactions.count++;

  transaction_timer = millis();

  purchased_energy = 0;
```

```
    //Serial.println("sent");
    if (debugging) {
      Serial.println("sent confirmation:");
      print_ping(&confirmation);
    }

    //update state and set reference current
    state = BUYING_FOR_USE;
    ref_current = -current_sensor_reading;
    Serial1.print(3);
    Serial1.print(terminator);
  }
}


void confirm_energy() {
  delay(30);
  char temp;
  temp = Serial1.read(); //get rid of the slash
  char node[3];
  Serial1.readBytesUntil('/', node, Serial1.available());
  //Serial.print(node);
  float qty;
  float prce;
  prce = Serial1.parseFloat() / 100 * battery_charge;
  //Serial.print(prce);
  Serial1.read(); // get rid of slash
  qty = Serial1.parseFloat();
  Serial1.read(); // get rid of slash
  //Serial.print(qty);
```

```
byte count;

count = Serial1.parseInt();

Serial1.read(); // get rid of slash

//Serial.print(count);

char path[5][3];

for (int i = 0; i < count + 1; i++) {

  Serial1.readBytes(path[i], 2);

  path[i][2] = '\0';

  //Serial.print(path[i]);

}

temp = Serial1.read();

temp = Serial1.read();


RF_ping data;

strcpy(data.destination, node);

data.type = 2;

data.count = count;

for (int i = 0; i < count + 1; i++) {

  strcpy(data.path[i], path[i]);

}

data.price = prce / (1 + count * 0.0029 * max_current + 0.1);

data.quantity = qty * (1 + count * 0.0029 * max_current + 0.1);


radio.stopListening();

persistant_radio_write(&data);

transaction_timer = millis();

if (debugging) {

  Serial.print(data.type);

  Serial.print("\n");

  Serial.print("writing confirmation \n");
```

```
    print_ping(&data);
  }
  radio.startListening();
  state = CONSUMING;
  ref_current = max_current;
  //switchbox_status[0] = 1; //this line will need to be generalized
  switchbox_status[find_index(data.path[1], neighbour_ids)] = 1;
  update_switchbox();
  transactions.list[transactions.count] = data;
  transactions.count ++;
  purchased_energy = 0;
  transaction_timer = millis();
}

void send_state_of_charge() {
  float percent = (battery_charge / battery_capacity) * 100;
  Serial1.write("1/");
  Serial1.print(percent);
  Serial1.print(terminator);
  if (debugging) {
    Serial.print("Sent SOC of charge to the phone: ");
    Serial.print(percent);
    Serial.print("\n");
  }
}

void receive_setup_data() {
  delay(30);
  char temp;
  //temp = Serial1.read(); //get rid of the slash
```

```
battery_capacity = Serial1.parseFloat();

battery_charge = max(min((converter_voltage_reading - 11.5) * 55.55, battery_capacity), 0);

max_current = battery_capacity / 3;

temp = Serial1.read(); //get rid of the slash

Serial1.readBytesUntil('/', node_id, Serial1.available());

Serial1.readBytesUntil('/', neighbour_ids[0], Serial1.available());

Serial1.readBytesUntil('/', neighbour_ids[1], Serial1.available());

Serial1.readBytesUntil('/', neighbour_ids[2], Serial1.available());

temp = Serial1.read(); //get rid of the terminator

if (debugging) {

  Serial.print(battery_capacity);

  Serial.print('\n');

  Serial.println(node_id);

  for (int i = 0; i < 3; i++) {

    Serial.print(neighbour_ids[i]);

    Serial.print('-');

  }

  Serial.print('\n');

}

radio.stopListening();

//addresses take format -0-sender-reader e.g. 01's reading pipe connected to 02 is 00201

addresses[0][0][1] = node_id[0];

addresses[0][0][2] = node_id[1];

addresses[0][0][3] = neighbour_ids[0][0];

addresses[0][0][4] = neighbour_ids[0][1];

addresses[0][1][3] = node_id[0];

addresses[0][1][4] = node_id[1];

addresses[0][1][1] = neighbour_ids[0][0];

addresses[0][1][2] = neighbour_ids[0][1];

addresses[1][0][1] = node_id[0];
```

```
  addresses[1][0][2] = node_id[1];

  addresses[1][0][3] = neighbour_ids[1][0];

  addresses[1][0][4] = neighbour_ids[1][1];

  addresses[1][1][3] = node_id[0];

  addresses[1][1][4] = node_id[1];

  addresses[1][1][1] = neighbour_ids[1][0];

  addresses[1][1][2] = neighbour_ids[1][1];

  addresses[2][0][1] = node_id[0];

  addresses[2][0][2] = node_id[1];

  addresses[2][0][3] = neighbour_ids[2][0];

  addresses[2][0][4] = neighbour_ids[2][1];

  addresses[2][1][3] = node_id[0];

  addresses[2][1][4] = node_id[1];

  addresses[2][1][1] = neighbour_ids[2][0];

  addresses[2][1][2] = neighbour_ids[2][1];


  if (debugging) {

    for (int i = 0; i < 2; i++) {

      for (int j = 0; j < 6; j++) {

        Serial.write(addresses[0][i][j]);

      }

      Serial.print('-');

    }

  }

  radio.openWritingPipe(addresses[0][0]);

  radio.openReadingPipe(1, addresses[0][1]);

  radio.startListening();

}

/**

 * 跟新开关信息
```

```
 * TODO 控制电路开关
 */
void update_switchbox() {
  /*
    This code just takes the info in the switchbox_status array and aligns the switchbox with it
  */
  for (int i = 0; i < 3; i++) {
    digitalWrite(SWITCHBOX[i], switchbox_status[i]);
  }
}


int find_index(const char ID[3], const char list[][3]) {
  // find index of ID in list, return index
  for (int i = 0; i < (sizeof(list) / sizeof(char)); i++) {
    if (String(ID).equals(String(list[i]))) {
      return i ;
    }
  }
  if (debugging) {
    Serial.print("invalid neighbouring ID \n");
  }
  return -1;
}


void flash() {
  for (int i = 0; i < 5; i++) {
    digitalWrite(LED_BUILTIN, HIGH);
    delay(200);
    digitalWrite(LED_BUILTIN, LOW);
    delay(200);
```

```
  }

}


void print_ping(RF_ping *data) {

  /*

    This just pretty-prints a RF ping object to the serial monitor

  */

  if (debugging) {

    Serial.print("destination: ");

    Serial.print((*data).destination);

    Serial.print("\n");

    Serial.print("type: ");

    Serial.print((*data).type);

    Serial.print("\n");

    Serial.print("count: ");

    Serial.print((*data).count);

    Serial.print("\n");

    Serial.print("path: ");

    for (int i = 0; i < 5; i++) {

      Serial.print((*data).path[i]);

      Serial.print("-");

    }

    Serial.print("\n");

    Serial.print("p - q: ");

    Serial.print((*data).price);

    Serial.print(" - ");

    Serial.print((*data).quantity);

    Serial.print(" | ");

    Serial.print("\n");

    Serial.print("\n");
```

```
  }
}

void receive_radio_test() {
  if (radio.available()) {
    bool test;
    radio.read(&test, sizeof(test));
    if (debugging) {
      Serial.print("test recieved");
    }
    if (test == 0)    {
      digitalWrite(LED_BUILTIN, LOW);
    }
    else if (test == 1)    {
      digitalWrite(LED_BUILTIN, HIGH);
    }
  }
}

bool persistant_radio_write(const RF_ping *data) {
  /*
     Will try to write a radio package multiple times if the first fails
  */
  if (!radio.write(data, sizeof(*data))) {
    return false;
  }
}

void check_transaction_state() {
  /*
```

```
  This function serves a couple of purposes.

  1. Updates transaction energy and battery charge values

  2. Cancels transaction if transaction energy is reached or battery is full

  3. Updates reference current if in load following mode

*/

if ((transactions.count == 0) or ((millis() - transaction_timer) < 10)) {

  return;

}


float time_delta = millis() - transaction_timer;

transaction_timer = millis();

float temp = (time_delta / (3600 * 1000)) * ref_current;

purchased_energy += (time_delta / (1000 * 3600)) * ref_current * (1 + 2 * 0.0029 * ref_current);

battery_charge += (time_delta / (1000 * 3600)) * ref_current * .9;


if (state == BUYING_FOR_USE) {

  ref_current = max(0, -current_sensor_reading);

}


if ((state == GENERATING) and (current_sensor_reading * converter_voltage_reading >= 350)) {

  cancel_purchase();

}


if (battery_charge >= battery_capacity) {

  //cancel if battery full

  if (debugging) {

    Serial.println("cancelling because battery full");

    Serial.print("Battery charge: ");

    Serial.println(battery_charge);

    Serial.print("Battery capacity: ");
```

```
    Serial.println(battery_capacity);
  }
  cancel_purchase();
}
else {
  //cancel if energy transfer has exceeded offered quantity
  for (int i = 0; i < transactions.count; i++) {
    if (String(transactions.list[i].path[0]).equals(node_id) && (purchased_energy > transactions.list[i].quantity)) {
      if (debugging) {
        Serial.println("cancelling because transaction energy exceeded");
        Serial.print("Energy drawn: ");
        Serial.println(purchased_energy);
        Serial.print("Available q: ");
        Serial.println(transactions.list[i].quantity);
      }
      transactions.list[i].type = CANCELLATION;
      strcpy(transactions.list[i].destination, transactions.list[i].path[1]);
      radio.stopListening();
      radio.openWritingPipe(addresses[find_index(transactions.list[i].path[1], neighbour_ids)][0]);
      bool success = 0;
      success = persistant_radio_write(&(transactions.list[i]));
      radio.startListening();
      switchbox_status[find_index(transactions.list[i].path[1], neighbour_ids)] = 1;
      update_switchbox();
      if (debugging) {
        Serial.print("Cancellation sent: \n");
        print_ping(&(transactions.list[i]));
      }


      for (int j = i; j < transactions.count; j++) {
```

```
        transactions.list[j] = transactions.list[j + 1];
      }

      transactions.count --;
      if (transactions.count == 0) {
        state = IDLE_STATE;
      }
      return;
    }
  }
 }

}

void update_switch_solar_energy(){
 float percent = (battery_charge / battery_capacity) * 100;
 if(percent >= 99.5){
   digitalWrite(SWITCHBOX_SOLAR, LOW);
 }else {
   digitalWrite(SWITCHBOX_SOLAR, HIGH);
 }
}

/* ##############################################################
   SETUP
  ############################################################## */

void setup() {
 Serial.begin(9600);
```

```
Serial1.begin(9600);

Serial2.begin(9600);

t = millis();

currentTimer = millis();

radio.begin();

radio.openWritingPipe(addresses[0][0]);

radio.openReadingPipe(1, addresses[0][1]); // open reading pipe with first neighbour

//radio.openReadingPipe(2, addresses[1][1]); // open reading pipe with second neighbour

//radio.openReadingPipe(3, addresses[2][1]); // open reading pipe with third neighbour

radio.setPALevel(RF24_PA_MIN);

radio.startListening();

digitalWrite(LED_BUILTIN, LOW);

pinMode(LED_BUILTIN, OUTPUT);

for (int i = 0; i < 3; i++) {

  pinMode(SWITCHBOX[i], OUTPUT);

}


delay(1001);

manageArduinoCommunications();

battery_charge = max(min((converter_voltage_reading - 11.5) * 55.55, battery_capacity), 0);

//flash();

if (debugging) {

  Serial.print("setup complete \n");

}

//buy_for_use();


//

//this part is for testing purposes - send a request out upon startup if no phone is available


/*radio.stopListening();
```

```
  strcpy(info.destination, "02");
  info.type = 0;
  info.count = 1;
  strcpy(info.path[0], node_id);
  if(debugging){Serial.print(info.type);
  Serial.print("\n");
  Serial.print("writing request \n");}
  persistant_radio_write(&info);
  //radio.write(&info,sizeof(info));
  print_ping(&info);
  radio.startListening();*/
}

int k = 0;
void loop() {

 if (radio.available()) {
  receive_radio();
 }

 if (Serial1.available() > 0) {
  receive_BT();
 }

 update_SOC(&current_sensor_reading);//更新电流，能量数据
 check_transaction_state();

 //were used for communication with the converter. This same interface was used on the Arduino board running the converter.
 //向转流器发送数据？
 manageArduinoCommunications();
```

```
//太阳能电池开关
if (k==10000){
  update_switch_solar_energy();
  k = 0;
}
k++;//10s check once




if (state != IDLE_STATE) {       //set built in light on when transaction ongoing
  digitalWrite(LED_BUILTIN, HIGH);
}
else {
  digitalWrite(LED_BUILTIN, LOW);
}


if ((Serial.available() > 0) and (debugging)) {
  Serial.read();         //print out transaction state if serial input received
  Serial.println(state);
  Serial.print("transactions ongoing: ");
  Serial.println(transactions.count);
  Serial.print("transaction energy: ");
  Serial.println(purchased_energy, 5);
  Serial.print("Battery charge: ");
  Serial.println(battery_charge, 5);
  Serial.print("battery_capacity: ");
  Serial.println(battery_capacity, 5);
  Serial.print("current_sensor_reading: ");
  Serial.println(current_sensor_reading, 5);
  Serial.print("converter_voltage_reading: ");
```

```
    Serial.println(converter_voltage_reading, 5);
  }
}


int mVperAmp = 100;
float RawValue = 0;
double CurrentSensorVoltage;
double CurrentSensorAmps;
float offset = 512; //need to reconsider offset value
double total = 0;
int i = 0;


void update_SOC(float *current_reading) {
  RawValue = analogRead(A7) - offset;
  total = total + RawValue;
  i ++;
  if ((millis() - currentTimer) > 2500) {
    double avg = total / i;
    i = 0;
    total = 0;
    currentTimer = millis();
    CurrentSensorVoltage = (avg / 1024.0) * 3300; // Gets you mV
    CurrentSensorAmps = (-(CurrentSensorVoltage) / mVperAmp);
    (*current_reading) = CurrentSensorAmps;
    /*Serial.print("Raw Value = " ); // shows pre-scaled value
      Serial.print(avg);
      Serial.print("\t mV = "); // shows the voltage measured
      Serial.print(CurrentSensorVoltage,3);
      Serial.print("\t Amps = "); // shows the voltage measured
```

```
    Serial.println(CurrentSensorAmps,3); */

   double Ah = (CurrentSensorAmps + converter_current_reading) * (2.5 / 3600);

   battery_charge = battery_charge + Ah;

   //Serial.println(battery_charge);

  }

}

float f1 = 0;

float f2 = 2;



void sendToArduino(float *a, float *b) {

  /*

    This function is to send two floating point numbers using the Serial2 port.

    Its output can be parsed by the readFromArduino function, also included in this file.

    Rembmer to include Serial2.begin(9600) in the setup function.



    a is a pointer to the first number to send

    b is a point to the second number to send

    10 bits of data are sent

  */

  Serial2.print(*a, 2);

  Serial2.print('/');

  Serial2.print(*b, 2);

  Serial2.print('/');

}



void readFromArduino(float *c, float *d) {

  /*

    This function is to parse two floating point numbers recieved over the Serial2 port.

    Its input is the data sent by the readFromArduino function, also included in this file.

    Rembmer to include Serial2.begin(9600) in the setup function.
```

```
    the first number recieved will be stored in a

    the first number recieved will be stored in b

    this function will only do anything if a full 10 bits of data are available.

  */

  if (Serial2.available() < 10) {

    return;

  }

  else {

    //Serial.print("reading");

    *c = Serial2.parseFloat();

    Serial2.read();          // get rid of the slash

    *d = Serial2.parseFloat();

    Serial2.read();          // get rid of the slash

  }

}




void manageArduinoCommunications() {

  /*

    This function should be called once per iteration of the main loop.

    Declare unsigned long t as a global variable, set equal to millis() in the setup function

  */

  //readFromArduino(&converter_voltage_reading,&converter_current_reading); commented out due to non-functional converter

  if ((millis() - t) > 100) {

    t = millis();

    //Set values for f1 and f2 here

    //Serial.print(f1);

    //Serial.print(f2);

    //Serial.print('\n');
```

```
    if (state == GENERATING) {
      f1 = 0;
      f2 = ref_voltage;
    }
    else if (state == CONSUMING) {
      f1 = 1;
      f2 = ref_current;
    }
    else if (state == IDLE_STATE) {
      f1 = 1;
      f2 = 0;
    }
    sendToArduino(&f1, &f2);
  }
}
```